9 Markov Chain Monte Carlo

In the twentieth century, scientists and engineers began publishing books of random numbers (FIGURE 9.1). For scientists from previous centuries, these books would have looked like madness. For most of Western history, chance has been a villain. In classical Rome, chance was personified by Fortuna, goddess of cruel fate, with her spinning wheel of (mis)fortune. Opposed to her sat Minerva, goddess of wisdom and understanding. Only the desperate would pray to Fortuna, while everyone implored Minerva for aid. Certainly science was the domain of Minerva, a realm with no useful role for Fortuna to play.

But by the twentieth century, Fortuna and Minerva had become collaborators. Now few of us are bewildered by the notion that an understanding of chance could help us acquire wisdom. Everything from weather forecasting to finance to evolutionary biology is dominated by the study of stochastic processes.[144] Researchers rely upon random numbers for the proper design of experiments. And mathematicians routinely make use of random inputs to compute specific outputs.

This chapter introduces one commonplace example of Fortuna and Minerva's cooperation: the estimation of posterior probability distributions using a stochastic process known as MARKOV CHAIN MONTE CARLO (MCMC). Unlike earlier chapters in this book, here we'll produce samples from the joint posterior without maximizing anything. Instead of having to lean on quadratic and other approximations of the shape of the posterior, now we'll be able to sample directly from the posterior without assuming a Gaussian, or any other, shape.

The cost of this power is that it may take much longer for our estimation to complete, and usually more work is required to specify the model as well. But the benefit is escaping the awkwardness of assuming multivariate normality. Equally important is the ability to directly estimate models, such as the generalized linear and multilevel models of later chapters. Such models routinely produce non-Gaussian posterior distributions, and sometimes they cannot be estimated at all with the techniques of earlier chapters.

The good news is that tools for building and inspecting MCMC estimates are getting better all the time. In this chapter you'll meet a convenient way to convert the `quap` formulas you've used so far into Markov chains. The engine that makes this possible is STAN (free and online at: mc-stan.org). Stan's creators describe it as "a probabilistic programming language implementing statistical inference." You won't be working directly in Stan to begin with—the `rethinking` package provides tools that hide it from you for now. But as you move on to more advanced techniques, you'll be able to generate Stan versions of the models you already understand. Then you can tinker with them and witness the power of a fully armed and operational Stan.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

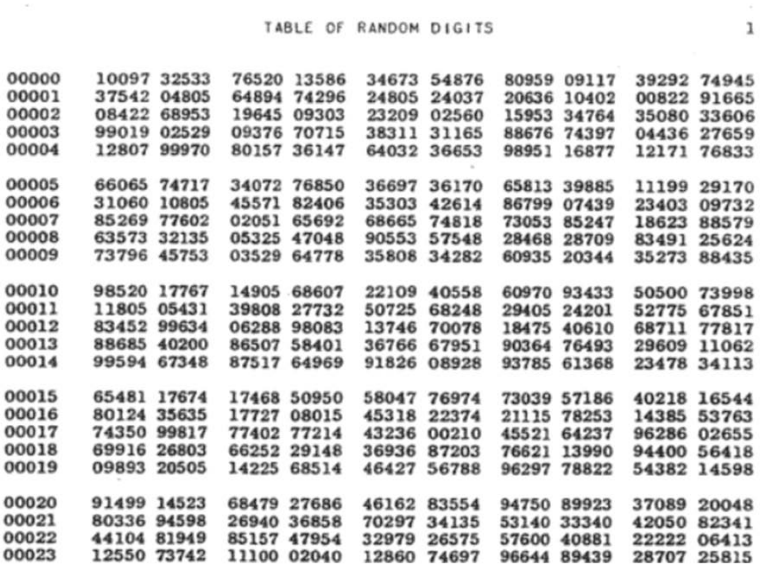


TABLE OF RANDOM DIGITS 1

00000	10097	32533	76520	13586	34673	54876	80959	09117	39292	74945
00001	37542	04805	64894	74296	24805	24037	20636	10402	00822	91665
00002	08422	68953	19645	09303	23209	02560	15953	34764	35080	33606
00003	99019	02529	09376	70715	38311	31165	88676	74397	04436	27659
00004	12807	99970	80157	36147	64032	36653	98951	16877	12171	76833
00005	66065	74717	34072	76850	36697	36170	65813	39885	11199	29170
00006	31060	10805	45571	82406	35303	42614	86799	07439	23403	09732
00007	85269	77602	02051	65692	68665	74818	73053	85247	18623	88579
00008	63573	32135	05325	47048	90553	57548	28468	28709	83491	25624
00009	73796	45753	03529	64778	35808	34282	60935	20344	35273	88435
00010	98520	17767	14905	68607	22109	40558	60970	93433	50500	73998
00011	11805	05431	39808	27732	50725	68248	29405	24201	52775	67851
00012	83452	99634	06288	98083	13746	70078	18475	40610	68711	77817
00013	88685	40200	86507	58401	36766	67951	90364	76493	29609	11062
00014	99594	67348	87517	64969	91826	08928	93785	61368	23478	34113
00015	65481	17674	17468	50950	58047	76974	73039	57186	40218	16544
00016	80124	35635	17727	08015	45318	22374	21115	78253	14385	53763
00017	74350	99817	77402	77214	43236	00210	45521	64237	96286	02655
00018	69916	26803	66252	29148	36936	87203	76621	13990	94400	56418
00019	09893	20505	14225	68514	46427	56788	96297	78822	54382	14598
00020	91499	14523	68479	27686	46162	83554	94750	89923	37089	20048
00021	80336	94598	26940	36858	70297	34135	53140	33340	42050	82341
00022	44104	81949	85157	47954	32979	26575	57600	40881	22222	06413
00023	12550	73742	11100	02040	12860	74697	96644	89439	28707	25815

FIGURE 9.1. A page from *A Million Random Digits*, a book consisting of nothing but random numbers.

Rethinking: Stan was a man. The Stan programming language is not an abbreviation or acronym. Rather, it is named after Stanisław Ulam (1909–1984). Ulam is credited as one of the inventors of Markov chain Monte Carlo. Together with Ed Teller, Ulam applied it to designing fusion bombs. But he and others soon applied the general Monte Carlo method to diverse problems of less monstrous nature. Ulam made important contributions in pure mathematics, chaos theory, and molecular and theoretical biology, as well.

9.1. Good King Markov and his island kingdom

For the moment, forget about posterior densities and MCMC. Consider instead the tale of Good King Markov.[145] King Markov was a benevolent autocrat of an island kingdom, a circular archipelago, with 10 islands. Each island was neighbored by two others, and the entire archipelago formed a ring. The islands were of different sizes, and so had different sized populations living on them. The second island was about twice as populous as the first, the third about three times as populous as the first, and so on, up to the largest island, which was 10 times as populous as the smallest.

The Good King was an autocrat, but he did have a number of obligations to his people. Among these obligations, King Markov agreed to visit each island in his kingdom from time to time. Since the people loved their king, each island preferred that he visit them more often. And so everyone agreed that the king should visit each island in proportion to its population size, visiting the largest island 10 times as often as the smallest, for example.

The Good King Markov, however, wasn't one for schedules or bookkeeping, and so he wanted a way to fulfill his obligation without planning his travels months ahead of time. Also, since the archipelago was a ring, the King insisted that he only move among adjacent islands,

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

to minimize time spent on the water—like many citizens of his kingdom, the king believed there were sea monsters in the middle of the archipelago.

The king's advisor, a Mr Metropolis, engineered a clever solution to these demands. We'll call this solution the *Metropolis algorithm*. Here's how it works.

(1) Wherever the King is, each week he decides between staying put for another week or moving to one of the two adjacent islands. To decide, he flips a coin.
(2) If the coin turns up heads, the King considers moving to the adjacent island clockwise around the archipelago. If the coin turns up tails, he considers instead moving counterclockwise. Call the island the coin nominates the *proposal* island.
(3) Now, to see whether or not he moves to the proposal island, King Markov counts out a number of seashells equal to the relative population size of the proposal island. So for example, if the proposal island is number 9, then he counts out 9 seashells. Then he also counts out a number of stones equal to the relative population of the current island. So for example, if the current island is number 10, then King Markov ends up holding 10 stones, in addition to the 9 seashells.
(4) When there are more seashells than stones, King Markov always moves to the proposal island. But if there are fewer shells than stones, he discards a number of stones equal to the number of shells. So for example, if there are 4 shells and 6 stones, he ends up with 4 shells and $6 - 4 = 2$ stones. Then he places the shells and the remaining stones in a bag. He reaches in and randomly pulls out one object. If it is a shell, he moves to the proposal island. Otherwise, he stays put another week. As a result, the probability that he moves is equal to the number of shells divided by the original number of stones.

This procedure may seem baroque and, honestly, a bit crazy. But it does work. The king will appear to move around the islands randomly, sometimes staying on one island for weeks, other times bouncing around without apparent pattern. But in the long run, this procedure guarantees that the king will be found on each island in proportion to its population size.

You can prove this to yourself, by simulating King Markov's journey. Here's a short piece of code to do this, storing the history of the king's journey in the vector `positions`:

R code 9.1

```
num_weeks <- 1e5
positions <- rep(0,num_weeks)
current <- 10
for ( i in 1:num_weeks ) {
  ## record current position
    positions[i] <- current
  ## flip coin to generate proposal
    proposal <- current + sample( c(-1,1) , size=1 )
  ## now make sure he loops around the archipelago
    if ( proposal < 1 ) proposal <- 10
    if ( proposal > 10 ) proposal <- 1
  ## move?
    prob_move <- proposal/current
    current <- ifelse( runif(1) < prob_move , proposal , current )
}
```

I've added comments to this code, to help you decipher it. The first three lines just define the number of weeks to simulate, an empty history vector, and a starting island position (the

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

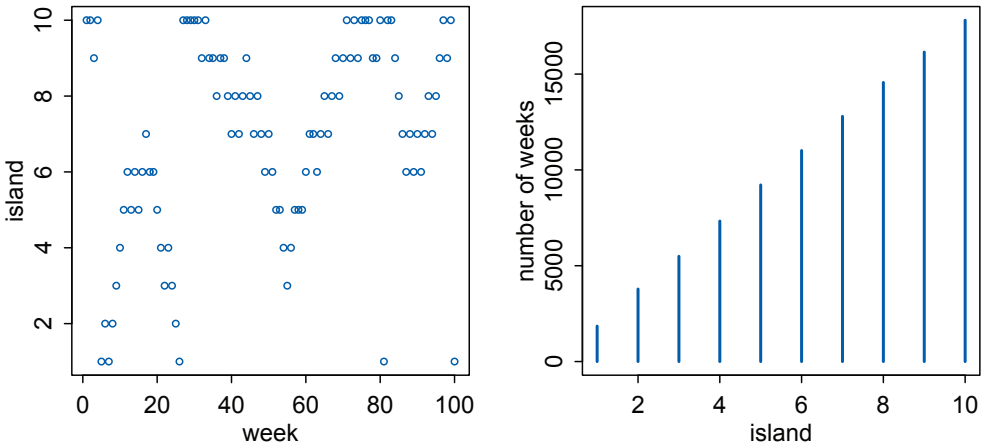


FIGURE 9.2. Results of the king following the Metropolis algorithm. The left-hand plot shows the king's position (vertical axis) across weeks (horizontal axis). In any particular week, it's nearly impossible to say where the king will be. The right-hand plot shows the long-run behavior of the algorithm, as the time spent on each island turns out to be proportional to its population size.

biggest island, number 10). Then the `for` loop steps through the weeks. Each week, it records the king's current position. Then it simulates a coin flip to nominate a proposal island. The only trick here lies in making sure that a proposal of "11" loops around to island 1 and a proposal of "0" loops around to island 10. Finally, a random number between zero and one is generated (`runif(1)`), and the king moves, if this random number is less than the ratio of the proposal island's population to the current island's population (`proposal/current`).

You can see the results of this simulation in FIGURE 9.2. The left-hand plot shows the king's location across the first 100 weeks of his simulated travels.

R code 9.2
```
plot( 1:100 , positions[1:100] )
```

As you move from the left to the right in this plot, the points show the king's location through time. The king travels among islands, or sometimes stays in place for a few weeks. This plot demonstrates the seemingly pointless path the Metropolis algorithm sends the king on. The right-hand plot shows that the path is far from pointless, however.

R code 9.3
```
plot( table( positions ) )
```

The horizontal axis is now islands (and their relative populations), while the vertical is the number of weeks the king is found on each. After the entire 100,000 weeks (almost 2000 years) of the simulation, you can see that the proportion of time spent on each island converges to be almost exactly proportional to the relative populations of the islands.

The algorithm will still work in this way, even if we allow the king to be equally likely to propose a move to any island from any island, not just among neighbors. As long as

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

King Markov still uses the ratio of the proposal island's population to the current island's population as his probability of moving, in the long run, he will spend the right amount of time on each island. The algorithm would also work for any size archipelago, even if the king didn't know how many islands were in it. All he needs to know at any point in time is the population of the current island and the population of the proposal island. Then, without any forward planning or backwards record keeping, King Markov can satisfy his royal obligation to visit his people proportionally.

9.2. Metropolis algorithms

The precise algorithm King Markov used is a special case of the general **Metropolis algorithm** from the real world.[146] And this algorithm is an example of Markov chain Monte Carlo. In real applications, the goal is of course not to help an autocrat schedule his journeys, but instead to draw samples from an unknown and usually complex target distribution, like a posterior probability distribution.

- The "islands" in our objective are parameter values, and they need not be discrete, but can instead take on a continuous range of values as usual.
- The "population sizes" in our objective are the posterior probabilities at each parameter value.
- The "weeks" in our objective are samples taken from the joint posterior of the parameters in the model.

Provided the way we choose our proposed parameter values at each step is symmetric—so that there is an equal chance of proposing from A to B and from B to A—then the Metropolis algorithm will eventually give us a collection of samples from the joint posterior. We can then use these samples just like all the samples you've already used in this book.

The Metropolis algorithm is the grandparent of several different strategies for getting samples from unknown posterior distributions. In the remainder of this section, I briefly explain the concept behind Gibbs sampling. Gibbs sampling is much better than plain Metropolis, and it continues to be common in applied Bayesian statistics. But it is rapidly being replaced by other algorithms.

9.2.1. Gibbs sampling. The Metropolis algorithm works whenever the probability of proposing a jump to B from A is equal to the probability of proposing A from B, when the proposal distribution is symmetric. There is a more general method, known as Metropolis-Hastings,[147] that allows asymmetric proposals. This would mean, in the context of King Markov's fable, that the King's coin were biased to lead him clockwise on average.

Why would we want an algorithm that allows asymmetric proposals? One reason is that it makes it easier to handle parameters, like standard deviations, that have boundaries at zero. A better reason, however, is that it allows us to generate savvy proposals that explore the posterior distribution more efficiently. By "more efficiently," I mean that we can acquire an equally good image of the posterior distribution in fewer steps.

The most common way to generate savvy proposals is a technique known as **Gibbs sampling**.[148] Gibbs sampling is a variant of the Metropolis-Hastings algorithm that uses clever proposals and is therefore more efficient. By "efficient," I mean that you can get a good estimate of the posterior from Gibbs sampling with many fewer samples than a comparable Metropolis approach. The improvement arises from *adaptive proposals* in which the distribution of proposed parameter values adjusts itself intelligently, depending upon the parameter values at the moment.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

How Gibbs sampling computes these adaptive proposals depends upon using particular combinations of prior distributions and likelihoods known as *conjugate pairs*. Conjugate pairs have analytical solutions for the posterior distribution of an individual parameter. And these solutions are what allow Gibbs sampling to make smart jumps around the joint posterior distribution of all parameters.

In practice, Gibbs sampling can be very efficient, and it's the basis of popular Bayesian model fitting software like `BUGS` (Bayesian inference Using Gibbs Sampling) and `JAGS` (Just Another Gibbs Sampler). In these programs, you compose your statistical model using definitions very similar to what you've been doing so far in this book. The software automates the rest, to the best of its ability.

9.2.2. High-dimensional problems. But there are some severe limitations to Gibbs sampling. First, maybe you don't want to use conjugate priors. Some conjugate priors are actually pathological in shape, once you start building multilevel models and need priors for entire covariance matrixes. This will be something to discuss once we reach Chapter 14.

Second, as models become more complex and contain hundreds or thousands or tens of thousands of parameters, both Metropolis and Gibbs sampling become shockingly inefficient. The reason is that they tend to get stuck in small regions of the posterior for potentially a long time. The high number of parameters isn't the problem so much as the fact that models with many parameters nearly always have regions of high correlation in the posterior. This means that two or more parameters are highly correlated with one another in the posterior samples. You've seen this before with, for example, the two legs example in Chapter 6. Why is this a problem? Because high correlation means a narrow ridge of high probability combinations, and both Metropolis and Gibbs make too many dumb proposals of where to go next. So they get stuck.

A picture will help to make this clearer. FIGURE 9.3 shows an ordinary Metropolis algorithm trying to explore a 2-dimensional posterior with a strong negative correlation of -0.9. The region of high-probability parameter values forms a narrow valley. Focus on the left-hand plot for now. The chain starts in the upper-left of the valley. Filled points are accepted proposals. Open points are rejected proposals. Proposals are generated by adding random Gaussian noise to each parameter, using a standard deviation of 0.01, the *step size*. 50 proposals are shown. The acceptance rate is only 60%, because when the valley is narrow like this, proposals can easily fall outside it. But the chain does manage to move slowly down the valley. It moves slow, because even when a proposal is accepted, it is still close to the previous point.

What happens then if we increase the step size, for more distant proposals? Now look on the right in FIGURE 9.3. Only 30% of proposals are accepted now. A bigger step size means more silly proposals outside the valley. The accepted proposals do move faster along the length of the valley, however. In practice, it is hard to win this tradeoff. Both Metropolis and Gibbs get stuck like this, because their proposals don't know enough about the global shape of the posterior. They don't know where they are going.

The high correlation example illustrates the problem. But the actual problem is more severe and more interesting. Any Markov chain approach that samples individual parameters in individual steps is going to get stuck, once the number of parameters grows sufficiently large. The reason goes by the name **CONCENTRATION OF MEASURE**. This is an awkward name for the amazing fact that most of the probability mass of a high-dimension distribution is always very far from the mode of the distribution. It is hard to visualize. We can't see in 100

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

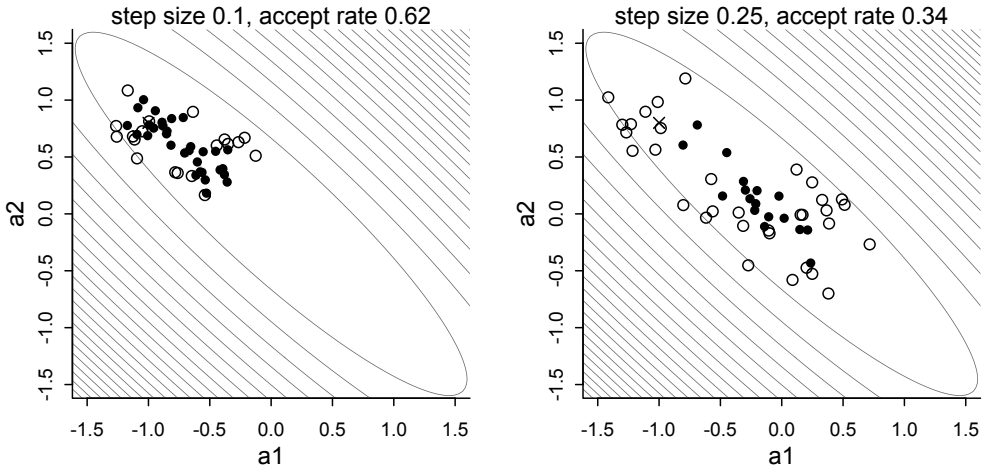


FIGURE 9.3. Metropolis chains under high correlation. Filled points indicate accepted proposals. Open points are rejected proposals. Both plots show 50 proposals under different proposal distribution step sizes. Left: With a small step size, the chain very slowly makes its way down the valley. It rejects 40% of the proposals in the process, because most of the proposals are in silly places. Right: With a larger step size, the chain moves faster, but it now rejects 70% of the proposals, because they tend to be even sillier. In higher dimensions, it is essentially impossible to tune Metropolis or Gibbs to be efficient.

dimensions, on most days. But if we think about the 2D and 3D versions, we can understand the basic phenomenon. In two dimensions, a Gaussian distribution is a hill. The highest point is in the middle, at the mode. But if we imagine this hill is filled with dirt—what else are hills filled with?—then we can ask: Where is most of the dirt? As we move away from the peak in any direction, the altitude declines, so there is less dirt directly under our feet. But in the ring around the hill at the same distance, there is more dirt than there is at the peak. The area increases as we move away from the peak, even though the height goes down. So the total dirt, um probability, increases as we move away from the peak. Eventually the total dirt (probability) declines again, as the hill slopes down to zero. So at some radial distance from the peak, dirt (probability mass) is maximized. In three dimensions, it isn't a hill, but now a fuzzy sphere. The sphere is densest at the core, its "peak." But again the volume increases as we move away from the core. So there is more total sphere-stuff in a shell around the core.

Back to thinking of probability distributions, all of this means that the combination of parameter values that maximizes posterior probability, the mode, is not actually in a region of parameter values that are highly plausible. This means in turn that when we properly sample from a high dimensional distribution, we won't get any points near the mode. You can demonstrate this for yourself very easily. Just sample randomly from a high-dimension distribution—10 dimensions is enough—and plot the radial distances of the points. Here's some code to do this:

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

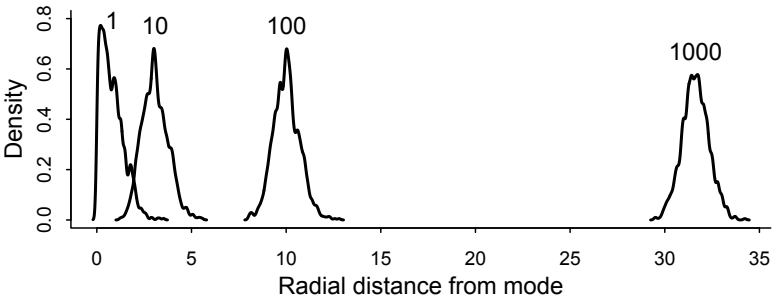


FIGURE 9.4. Concentration of measure and the curse of high dimensions. The horizontal axis shows radial distance from the mode in parameter space. Each density is a random sample of 1000 points. The number above each density is the number of dimensions. As the number of parameters increases, the mode is further away from the values we want to sample.

R code 9.4

```
D <- 10
T <- 1e3
Y <- rmvnorm(T,rep(0,D),diag(D))
rad_dist <- function( Y ) sqrt( sum(Y^2) )
Rd <- sapply( 1:T , function(i) rad_dist( Y[i,] ) )
dens( Rd )
```

I display this density, as well as the corresponding densities for distributions with 1, 100, and 1000 dimensions, in FIGURE 9.4. The horizontal axis here is radial distance of the point from the mode. So the value 0 is the peak of probability. You can see that an ordinary Gaussian distribution with only 1 dimension, on the left, samples most of its points right next to this peak, as you'd expect. But with 10 dimensions, already there are no samples next to the peak at zero. With 100 dimensions, we've moved very far from the peak. And with 1000 dimensions, even further. The sampled points are in a thin, high-dimensional shell very far from the mode. This shell can create very hard paths for a sampler to follow.

This is why we need MCMC algorithms that focus on the entire posterior at once, instead of one or a few dimensions at a time like Metropolis and Gibbs. Otherwise we get stuck in a narrow, highly curving region of parameter space.

9.3. Hamiltonian Monte Carlo

> It appears to be a quite general principle that, whenever there is a randomized way of doing something, then there is a nonrandomized way that delivers better performance but requires more thought. —E. T. Jaynes

The Metropolis algorithm and Gibbs sampling are highly random procedures. They try out new parameter values—proposals—and see how good they are, compared to the current values. Gibbs sampling gains efficiency by reducing the randomness of proposals by exploiting knowledge of the target distribution. This seems to fit Jaynes' suggestion, quoted above, that when there is a random way of accomplishing some calculation, there is probably a less

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

random way that is better.[149] This less random way may require a lot more thought. The Gibbs strategy has limitations, but it gets its improvement over plain Metropolis by being less random, not more.

Hamiltonian Monte Carlo (or Hybrid Monte Carlo, HMC) pushes Jaynes' principle much further. HMC is more computationally costly than Metropolis or Gibbs sampling. But its proposals are also much more efficient. As a result, HMC doesn't need as many samples to describe the posterior distribution. You need less computer time in total, even though each sample needs more. And as models become more complex—thousands or tens of thousands of parameters—HMC can really outshine other algorithms, because the other algorithms just won't work. The Earth would be swallowed by the Sun before your chain produces a reliable approximation of the posterior.

We're going to be using HMC on and off for the remainder of this book. You won't have to implement it yourself. But understanding some of the concept behind it will help you grasp how it outperforms Metropolis and Gibbs sampling and also how it encounters its own, unique problems.

9.3.1. Another parable. Suppose King Markov's cousin Monty is King on the mainland. Monty's kingdom is not a discrete set of islands. Instead, it is a continuous territory stretched out along a narrow valley, running north-south. But the King has a similar obligation: to visit his citizens in proportion to their local population density. Within the valley, people distribute themselves inversely proportional to elevation—most people live in the middle of the valley, fewer up the mountainside. How can King Monty fulfill his royal obligation?

Like Markov, Monty doesn't wish to bother with schedules and calculations. Also like Markov, Monty has a highly educated and mathematically gifted advisor, named Hamilton. Hamilton designed an odd, but highly efficient, method. And this method solves one of Metropolis' flaws—the king hardly ever stays in the same place, but keeps moving on to visit new locations.

Here's how it works. The king's vehicle picks a random direction, either north or south, and drives off at a random momentum. As the vehicle goes uphill, it slows down and turns around when its declining momentum forces it to. Then it picks up speed again on the way down. After a fixed period of time, they stop the vehicle, get out, and start shaking hands and kissing babies. Then they get back in the vehicle and begin again. Amazingly, Hamilton can prove mathematically that this procedure guarantees that, in the long run, the locations visited will be inversely proportional to their relative elevations, which are also inversely proportional to the population densities. Not only does this keep the king moving, but it also spaces the locations apart better—unlike the other king, Monty does not only visit neighboring locations.

This mad plan is illustrated, and simulated, in Figure 9.5. The horizontal axis is time. The vertical axis is location. The king's journey starts on the far left, in the middle of the valley. The vehicle begins by heading south. The width of the curve indicates the momentum at each time. The vehicle climbs uphill but slows and briefly turns around before stopping at the first location. Then again and again new locations are chosen in the same way, but with different random directions and momentums, departing from the most recent location. When the initial momentum is small, the vehicle starts to turn around earlier. But when the initial momentum is large, like in the big swing around time 300, the king can traverse the entire valley before stopping.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

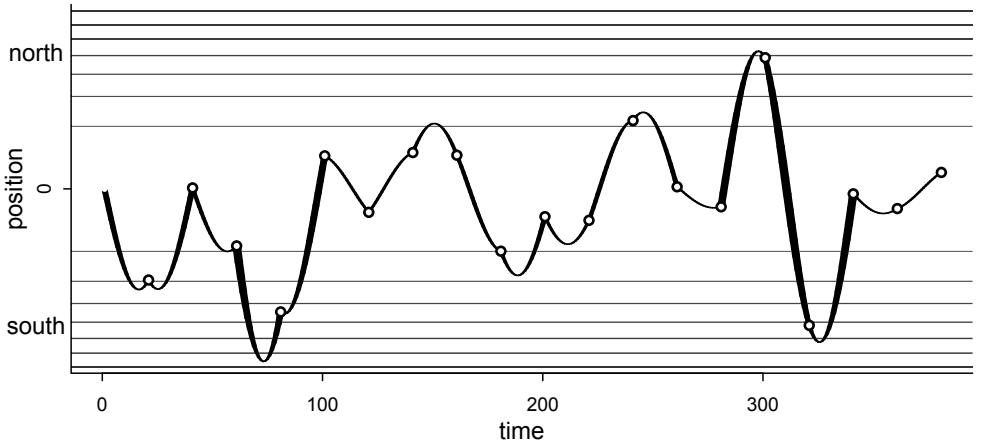


FIGURE 9.5. King Monty's Royal Drive. The journey begins at time 1 on the far left. The vehicle is given a random momentum and a random direction, either north (top) or south (bottom). The thickness of the path shows momentum at each moment. The vehicle travels, losing momentum uphill or gaining it downhill. After a fixed amount of time, they stop and make a visit, as shown by the points. Then a new random direction and momentum is chosen. In the long run, positions are visited in proportion to their population density.

The **AUTOCORRELATION** between locations visited is very low under this strategy. This means that adjacent locations have a very low, almost zero correlation. The king can move from one end of the valley to another. This stands in contrast to the highly autocorrelated movement under the Metropolis plan (FIGURE 9.2). King Markov of the Islands might wish to adopt this Hamiltonian strategy, but he cannot: The islands are not continuous. Hamilton's approach only works when all the locations are connected by dry land, because it requires that the vehicle be capable of stopping at any point.

Rethinking: Hamiltonians. The Hamilton who gives his name to Hamiltonian Monte Carlo had nothing to do with the development of the method. Sir William Rowan Hamilton (1805–1865) was an Irish mathematician, arguably the greatest mathematician of his generation. Hamilton accomplished great things in pure mathematics, but he also dabbled in physics and reformulated Newton's laws of motion into a new system that we now call Hamiltonian mechanics (or dynamics). Hamiltonian Monte Carlo was originally called **HYBRID MONTE CARLO**, but is now usually referred to by the Hamiltonian differential equations that drive it.

9.3.2. Particles in space. This story of King Monty is analogous to how the actual Hamiltonian Monte Carlo algorithm works. In statistical applications, the royal vehicle is the current vector of parameter values. Let's consider the single parameter case, just to keep things simple. In that case, the log-posterior is like a bowl, with the point of highest posterior probability at its nadir, in the center of the valley. Then we give the particle a random flick—give

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

it some momentum—and simulate its path. It must obey the physics, gliding along until we stop the clock and take a sample.

This is not another metaphor. HMC really does run a physics simulation, pretending the vector of parameters gives the position of a little frictionless particle. The log-posterior provides a surface for this particle to glide across. When the log-posterior is very flat, because there isn't much information in the likelihood and the priors are rather flat, then the particle can glide for a long time before the slope (gradient) makes it turn around. When instead the log-posterior is very steep, because either the likelihood or the priors are very concentrated, then the particle doesn't get far before turning around.

In principle, HMC will always accept every proposal, because it only makes intelligent proposals. In practice, HMC uses a rejection criterion, because it is only approximating the smooth path of a particle. It isn't unusual to see acceptance rates over 95% with HMC. Making smart proposals pays. What is the rejection criterion? Because HMC runs a physics simulation, certain things have to be conserved, like total energy of the system. When the total energy changes during the simulation, that means the numerical approximation is bad. When the approximation isn't good, it might reject the proposal.

All of this sounds, and is, complex. But what is gained from all of this complexity is very efficient sampling of complex models. In cases where ordinary Metropolis or Gibbs sampling wander slowly through parameter space, Hamiltonian Monte Carlo remains efficient. This is especially true when working with multilevel models with hundreds or thousands of parameters. A particle in 1000-dimension space sounds crazy, but it's not harder for your compute to imagine than a particle in 3-dimensions.

To take some of the magic out of this, let's do a two-dimensional simulation, for a simple posterior distribution with two parameters, the mean and standard deviation of a Gaussian. I'm going to show just the most minimal mathematical details. You don't need to grasp all the mathematics to make use of HMC. But having some intuition about how it works will help you appreciate why it works so much better than other approaches, as well as why it sometimes doesn't work. If you want much more mathematical detail, follow the endnote.[150]

Suppose the data are 100 x and 100 y values, all sampled from Normal$(0, 1)$. We'll use this statistical model:

$$\begin{aligned} x_i &\sim \text{Normal}(\mu_x, 1) \\ y_i &\sim \text{Normal}(\mu_y, 1) \\ \mu_x &\sim \text{Normal}(0, 0.5) \\ \mu_y &\sim \text{Normal}(0, 0.5) \end{aligned}$$

What HMC needs to drive are two functions and two settings. The first function computes the log-probability of the data and parameters. This is just the top part of Bayes' formula, and every MCMC strategy requires this. It tells the algorithm the "elevation" of any set of parameter values. For the model above, it is just:

$$\sum_i \log p(y_i|\mu_y, 1) + \sum_i \log p(x_i|\mu_x, 1) + \log p(\mu_y|0, 0.5) + \log p(\mu_x, 0, 0.5)$$

where $p(x|a, b)$ here means the Gaussian density of x at mean a and standard deviation b. The second thing HMC needs is the **GRADIENT**, which just means the slope in all directions at the current position. In this case, that means just two derivatives. If you take the expression above and differentiate it with respect to μ_x and then μ_y, you have what you need. I've placed

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

these derivatives, in code form, in the Overthinking box further down, where you'll find a complete R implementation of this example.

The two settings that HMC needs are a choice of number of **leapfrog steps** and a choice of **step size** for each. This part is strange. And usually your machine will pick these values for you. But having some idea of them will be useful for understanding some of the newer features of HMC algorithms. Each path in the simulation—each curve for example between visits in Figure 9.5—is divided up into a number of leapfrog steps. If you choose many steps, the paths will be long. If you choose few, they will be short. The size of each step is determined by, you guessed it, the step size. The step size determines how fine grained the simulation is. If the step size is small, then the particle can turn sharply. If the step size is large, then each leap will be large and could even overshoot the point where the simulation would want to turn around.

Let's put it all together in Figure 9.6. The code to reproduce this figure is in the Overthinking box below. The left simulation uses $L = 11$ leapfrog steps, each with a step size of $\epsilon = 0.03$. The contours show the log-posterior. It's a symmetric bowl in this example. Only 4 samples from the posterior distribution are shown. The chain begins at the $\times$. The first simulation gets flicked to the right and rolls downhill and then uphill again, stopping on the other side and taking a sample at the point labeled 1. The width of the path shows the total momentum, the kinetic energy, at each point.[151] Each leapfrog step is indicated by the white dots along the path. The process repeats, with random direction and momentum in both dimensions each time. You could take 100 samples here and get an excellent approximation with very low autocorrelation.

However, that low autocorrelation is not automatic. The right-hand plot in Figure 9.6 shows the same code but with $L = 28$ leapfrog steps. Now because of the combination of leapfrog steps and step size, the paths tend to land close to where they started. Instead of independent samples from the posterior, we get correlated samples, like in a Metropolis chain. This problem is called the **U-turn** problem—the simulations turn around and return to the same neighborhood. The U-turn problem looks especially bad in this example, because the posterior is a perfect 2-dimensional Gaussian bowl. So the parabolic paths always loop back onto themselves. In most models, this won't be the case. But you'll still get paths returning close to where they started. This just shows that the efficiency of HMC comes with the expense of having to tune the leapfrog steps and step size in each application.

Fancy HMC samplers, like Stan and its `rstan` package, have two ways to deal with U-turns. First, they will choose the leapfrog steps and step size for you. They can do this by conducting a **warmup** phase in which they try to figure out which step size explores the posterior efficiently. If you are familiar with older algorithms like Gibbs sampling, which use a burn-in phase, warmup is not like burn-in. Technically, burn-in samples are just samples. They are part of the posterior. But Stan's warmup phase, for example, does not produce useful samples. It is just tuning the simulation. The warmup phase tends to be slower than the sampling phase. So when you start using Stan, and warmup seems to be slow, in most cases it will speed up a lot as time goes on.

The second thing fancy HMC samplers do is use a clever algorithm to adaptively set the number of leapfrog steps. The type of algorithm is a **no-U-turn sampler**, or **NUTS**. A no-U-turn sampler uses the shape of the posterior to infer when the path is turning around. Then it stops the simulation. The details are both complicated and amazing.[152] Stan currently (since version 2.0) uses a second-generation NUTS2 sampler. See the Stan manual for more.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

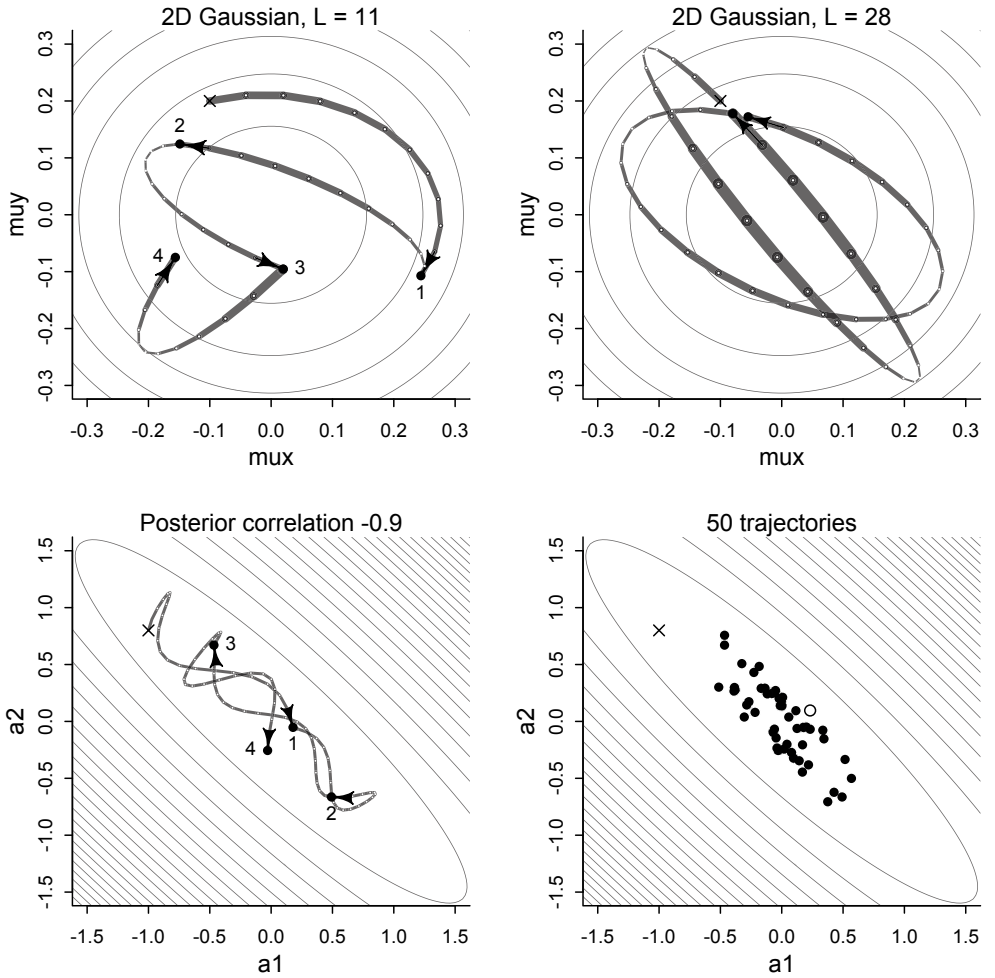


FIGURE 9.6. Hamiltonian Monte Carlo trajectories follow physical paths determined by the curvature of the posterior distribution. Top-left: With the right combination of leapfrog steps and step size, the individual paths produce independent samples from the posterior. The simulation begins at the $\times$ and then moves in order to the points labeled 1 though 4. Top-right: With the wrong combination, sequential samples can end up very close to one another. The chain in the top-right will still work. It'll just be much less efficient. Bottom-left: HMC really shines when the posterior contains high correlations, as here. Bottom-right: 50 samples from the same high correlation posterior, showing only one rejected sample (the open point). A low rate of rejected proposals and lower autocorrelation between samples means fewer samples are needed to approximate the posterior.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

Overthinking: Hamiltonian Monte Carlo in the raw. The HMC algorithm needs five things to go: (1) a function `U` that returns the negative log-probability of the data at the current position (parameter values), (2) a function `grad_U` that returns the *gradient* of the negative log-probability at the current position, (3) a step size `epsilon`, (4) a count of leapfrog steps `L`, and (5) a starting position `current_q`. Keep in mind that the position is a vector of parameter values and that the gradient also needs to return a vector of the same length. So that these `U` and `grad_U` functions make more sense, let's present them first, built custom for the 2D Gaussian example. The `U` function just expresses the log-posterior, as stated before in the main text:

$$\sum_i \log p(y_i|\mu_y, 1) + \sum_i \log p(x_i|\mu_x, 1) + \log p(\mu_y|0, 0.5) + \log p(\mu_x, 0, 0.5)$$

So it's just four calls to `dnorm` really:

R code 9.5

```
# U needs to return neg-log-probability
U <- function( q , a=0 , b=1 , k=0 , d=1 ) {
    muy <- q[1]
    mux <- q[2]
    U <- sum( dnorm(y,muy,1,log=TRUE) ) + sum( dnorm(x,mux,1,log=TRUE) ) +
        dnorm(muy,a,b,log=TRUE) + dnorm(mux,k,d,log=TRUE)
    return( -U )
}
```

Now the gradient function requires two partial derivatives. Luckily, Gaussian derivatives are very clean. The derivative of the logarithm of any univariate Gaussian with mean a and standard deviation b with respect to a is:

$$\frac{\partial \log \mathrm{N}(y|a, b)}{\partial a} = \frac{y - a}{b^2}$$

And since the derivative of a sum is a sum of derivatives, this is all we need to write the gradients:

$$\frac{\partial U}{\partial \mu_x} = \frac{\partial \log \mathrm{N}(x|\mu_x, 1)}{\partial \mu_x} + \frac{\partial \log \mathrm{N}(\mu_x|0, 0.5)}{\partial \mu_x} = \sum_i \frac{x_i - \mu_x}{1^2} + \frac{0 - \mu_x}{0.5^2}$$

And the gradient for μ_y has the same form. Now in code form:

R code 9.6

```
# gradient function
# need vector of partial derivatives of U with respect to vector q
U_gradient <- function( q , a=0 , b=1 , k=0 , d=1 ) {
    muy <- q[1]
    mux <- q[2]
    G1 <- sum( y - muy ) + (a - muy)/b^2 #dU/dmuy
    G2 <- sum( x - mux ) + (k - mux)/d^2 #dU/dmux
    return( c( -G1 , -G2 ) ) # negative bc energy is neg-log-prob
}
# test data
set.seed(7)
y <- rnorm(50)
x <- rnorm(50)
x <- as.numeric(scale(x))
y <- as.numeric(scale(y))
```

The gradient function above isn't too bad for this model. But it can be terrifying for a reasonably complex model. That is why tools like Stan build the gradients dynamically, using the model definition. Now we are ready to visit the heart. To understand some of the details here, you should read Radford Neal's chapter in the *Handbook of Markov Chain Monte Carlo*. Armed with the log-posterior and gradient functions, here's the code to produce FIGURE 9.6:

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

R code 9.7

```
library(shape) # for fancy arrows
Q <- list()
Q$q <- c(-0.1,0.2)
pr <- 0.3
plot( NULL , ylab="muy" , xlab="mux" , xlim=c(-pr,pr) , ylim=c(-pr,pr) )
step <- 0.03
L <- 11 # 0.03/28 for U-turns --- 11 for working example
n_samples <- 4
path_col <- col.alpha("black",0.5)
points( Q$q[1] , Q$q[2] , pch=4 , col="black" )
for ( i in 1:n_samples ) {
    Q <- HMC2( U , U_gradient , step , L , Q$q )
    if ( n_samples < 10 ) {
      for ( j in 1:L ) {
        K0 <- sum(Q$ptraj[j,]^2)/2 # kinetic energy
        lines( Q$traj[j:(j+1),1] , Q$traj[j:(j+1),2] , col=path_col , lwd=1+2*K0 )
      }
      points( Q$traj[1:L+1,] , pch=16 , col="white" , cex=0.35 )
      Arrows( Q$traj[L,1] , Q$traj[L,2] , Q$traj[L+1,1] , Q$traj[L+1,2] ,
          arr.length=0.35 , arr.adj = 0.7 )
      text( Q$traj[L+1,1] , Q$traj[L+1,2] , i , cex=0.8 , pos=4 , offset=0.4 )
    }
    points( Q$traj[L+1,1] , Q$traj[L+1,2] , pch=ifelse( Q$accept==1 , 16 , 1 ) ,
        col=ifelse( abs(Q$dH)>0.1 , "red" , "black" ) )
}
```

The function `HMC2` is built into `rethinking`. It is based upon one of Radford Neal's example scripts.[153] It isn't actually too complicated. Let's tour through it, one step at a time, to take the magic away. This function runs a single trajectory, and so produces a single sample. You need to use it repeatedly to build a chain. That's what the loop above does. The first chunk of the function chooses random momentum—the flick of the particle—and initializes the trajectory.

R code 9.8

```
HMC2 <- function (U, grad_U, epsilon, L, current_q) {
  q = current_q
  p = rnorm(length(q),0,1) # random flick - p is momentum.
  current_p = p
  # Make a half step for momentum at the beginning
  p = p - epsilon * grad_U(q) / 2
  # initialize bookkeeping - saves trajectory
  qtraj <- matrix(NA,nrow=L+1,ncol=length(q))
  ptraj <- qtraj
  qtraj[1,] <- current_q
  ptraj[1,] <- p
```

Then the action comes in a loop over leapfrog steps. `L` steps are taken, using the gradient to compute a linear approximation of the log-posterior surface at each point.

R code 9.9

```
  # Alternate full steps for position and momentum
  for ( i in 1:L ) {
    q = q + epsilon * p # Full step for the position
    # Make a full step for the momentum, except at end of trajectory
    if ( i!=L ) {
        p = p - epsilon * grad_U(q)
        ptraj[i+1,] <- p
    }
```

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

```
    qtraj[i+1,] <- q
  }
```

Notice how the step size `epsilon` is added to the position and momentum vectors. It is in this way that the path is only an approximation, because it is a series of linear jumps, not an actual smooth curve. This can have important consequences, if the log-posterior bends sharply and the simulation jumps over a bend. All that remains is clean up: ensure the proposal is symmetric so the Markov chain is valid and decide whether to accept or reject the proposal.

R code 9.10

```
  # Make a half step for momentum at the end
  p = p - epsilon * grad_U(q) / 2
  ptraj[L+1,] <- p
  # Negate momentum at end of trajectory to make the proposal symmetric
  p = -p
  # Evaluate potential and kinetic energies at start and end of trajectory
  current_U = U(current_q)
  current_K = sum(current_p^2) / 2
  proposed_U = U(q)
  proposed_K = sum(p^2) / 2
  # Accept or reject the state at end of trajectory, returning either
  # the position at the end of the trajectory or the initial position
  accept <- 0
  if (runif(1) < exp(current_U-proposed_U+current_K-proposed_K)) {
    new_q <- q  # accept
    accept <- 1
  } else new_q <- current_q  # reject
  return(list( q=new_q, traj=qtraj, ptraj=ptraj, accept=accept ))
}
```

The accept/reject decision at the bottom uses the fact that in Hamiltonian dynamics, the total energy of the system must be constant. So if the energy at the start of the trajectory differs substantially from the energy at the end, something has gone wrong. This is known as a **DIVERGENT TRANSITION**, and we'll talk more about these in a later chapter.

9.3.3. Limitations. As always, there are some limitations. HMC requires continuous parameters. It can't glide through a discrete parameter. In practice, this means that certain techniques, like the imputation of discrete missing data, have to be done differently with HMC. HMC can certainly sample from such models, often much more efficiently than a Gibbs sampler could. But you have to change how you code them. There will be examples in Chapter 15 and Chapter 16.

It is also important to keep in mind that HMC is not magic. Some posterior distributions are just very difficult to sample from, for any algorithm. We'll see examples in later chapters. In these cases, HMC will encounter something called a **DIVERGENT TRANSITION**. We'll talk a lot about these, what causes them, and how to fix them, later on.

Rethinking: The MCMC horizon. While the ideas behind Markov chain Monte Carlo are not new, widespread use dates only to the last decade of the twentieth century.[154] New variants of and improvements to MCMC algorithms arise all the time. We might anticipate that interesting advances are coming, and that the current crop of tools—Gibbs sampling and first-generation HMC for example—will look rather pedestrian in another 20 years. At least we can hope.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

9.4. Easy HMC: `ulam`

The `rethinking` package provides a convenient interface, `ulam`, to compile lists of formulas, like the lists you've been using so far to construct `quap` estimates, into Stan HMC code. A little more housekeeping is needed to use `ulam`: You should preprocess any variable transformations, and you should construct a clean data list with only the variables you will use. But otherwise installing Stan on your computer is the hardest part. And once you get comfortable with interpreting samples produced in this way, you go peek inside and see exactly how the model formulas you already understand correspond to the code that drives the Markov chain. When you use `ulam`, you can also use the same helper functions as `quap`: `extract.samples`, `extract.prior`, `link`, `sim`, and others.

There are other R packages that make using Stan even easier, because they don't require the full formulas that `quap` and `ulam` do. At the time of printing, the best are `brms` and `rstanarm` for multilevel models and `blavaan` for structural equation models. For learning about Bayesian modeling, I recommend you stick with the full and explicit formulas of `ulam` for now. The reason is that an interface that hides the model structure makes it hard to learn the model structure. But there is nothing wrong with moving on to simplified interfaces later, once you gain experience.

To see how `ulam` works, let's revisit the terrain ruggedness example from Chapter 8. This code will load the data and reduce it down to cases (nations) that have the outcome variable of interest:

R code 9.11

```
library(rethinking)
data(rugged)
d <- rugged
d$log_gdp <- log(d$rgdppc_2000)
dd <- d[ complete.cases(d$rgdppc_2000) , ]
dd$log_gdp_std <- dd$log_gdp / mean(dd$log_gdp)
dd$rugged_std <- dd$rugged / max(dd$rugged)
dd$cid <- ifelse( dd$cont_africa==1 , 1 , 2 )
```

So you remember the old way, we're going to repeat the procedure for fitting the interaction model. This model aims to predict log GDP with terrain ruggedness, continent, and the interaction of the two. Here's the way to do it with `quap`, just like before.

R code 9.12

```
m8.3 <- quap(
    alist(
        log_gdp_std ~ dnorm( mu , sigma ) ,
        mu <- a[cid] + b[cid]*( rugged_std - 0.215 ) ,
        a[cid] ~ dnorm( 1 , 0.1 ) ,
        b[cid] ~ dnorm( 0 , 0.3 ) ,
        sigma ~ dexp( 1 )
    ) , data=dd )
precis( m8.3 , depth=2 )
```

```
       mean   sd  5.5% 94.5%
a[1]   0.89 0.02  0.86  0.91
a[2]   1.05 0.01  1.03  1.07
b[1]   0.13 0.07  0.01  0.25
```

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

```
b[2]  -0.14 0.05 -0.23 -0.06
sigma  0.11 0.01  0.10  0.12
```

Just as you saw in the previous chapter.

9.4.1. Preparation. But now we'll also fit this model using Hamiltonian Monte Carlo. This means there will be no more quadratic approximation—if the posterior distribution is non-Gaussian, then we'll get whatever non-Gaussian shape it has. You can use exactly the same formula list as before, but you should do two additional things.

(1) Preprocess all variable transformations. If the outcome is transformed somehow, like by taking the logarithm, then do this before fitting the model by constructing a new variable in the data frame. Likewise, if any predictor variables are transformed, including squaring and cubing and such to build polynomial models, then compute these transformed values before fitting the model. It's a waste of computing power to do these transformations repeatedly in every step of the Markov chain.

(2) Once you've got all the variables ready, make a new trimmed down data frame that contains only the variables you will actually use to fit the model. Technically, you don't have to do this. But doing so avoids common problems. For example, if any of the unused variables have missing values, `NA`, then Stan will refuse to work.

We've already pre-transformed all the variables. Now we need a slim list of the variables we will use:

R code 9.13

```
dat_slim <- list(
    log_gdp_std = dd$log_gdp_std,
    rugged_std = dd$rugged_std,
    cid = as.integer( dd$cid )
)
str(dat_slim)
```

```
List of 3
 $ log_gdp_std: num [1:170] 0.88 0.965 1.166 1.104 0.915 ...
 $ rugged_std : num [1:170] 0.138 0.553 0.124 0.125 0.433 ...
 $ cid        : int [1:170] 1 2 2 2 2 2 2 2 1 ...
```

It is better to use a `list` than a `data.frame`, because the elements in a `list` can be any length. In a `data.frame`, all the elements must be the same length. With some models to come later, like multilevel models, it isn't unusual to have variables of different lengths.

9.4.2. Sampling from the posterior. Now provided you have the `rstan` package installed (mc-stan.org), you can get samples from the posterior distribution with this code:

R code 9.14

```
m9.1 <- ulam(
    alist(
        log_gdp_std ~ dnorm( mu , sigma ) ,
        mu <- a[cid] + b[cid]*( rugged_std - 0.215 ) ,
        a[cid] ~ dnorm( 1 , 0.1 ) ,
        b[cid] ~ dnorm( 0 , 0.3 ) ,
        sigma ~ dexp( 1 )
    ) , data=dat_slim , chains=1 )
```

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

All that `ulam` does is translate the formula above into a Stan model, and then Stan defines the sampler and does the hard part. Stan models look very similar, but require some more explicit definitions. This also makes them much more flexible. If you'd rather start working directly with Stan code, I'll present this same model in raw Stan a bit later. You can always extract the Stan code with `stancode(m9.1)`.

After messages about compiling, and sampling, `ulam` returns an object that contains a bunch of summary information, as well as samples from the posterior distribution. You can summarize just like a `quap` model:

R code 9.15

```
precis( m9.1 , depth=2 )
```

```
       mean   sd  5.5% 94.5% n_eff Rhat4
a[1]   0.89 0.02  0.86  0.91   739     1
a[2]   1.05 0.01  1.03  1.07   714     1
b[1]   0.13 0.08  0.01  0.26   793     1
b[2]  -0.14 0.05 -0.23 -0.06   799     1
sigma  0.11 0.01  0.10  0.12   785     1
```

These estimates are very similar to the quadratic approximation. But note that there are two new columns, `n_eff` and `Rhat4`. These columns provide MCMC diagnostic criteria, to help you tell how well the sampling worked. We'll discuss them in detail later in the chapter. For now, it's enough to know that `n_eff` is a crude estimate of the number of independent samples you managed to get. Rhat ($\hat{R}$) is an indicator of the convergence of the Markov chains to the target distribution. It should approach 1.00 from above, when all is well. There are several different ways to compute it. The "4" on the end indicates the fourth generation version of Rhat, not the original 1992 version that you usually see cited in papers. In the future, this will increase to `Rhat5`, the 5th generation. See the details and citations in `?precis`.

9.4.3. Sampling again, in parallel. The example so far is a very easy problem for MCMC. So even the default 1000 samples is enough for accurate inference. In fact, as few as 200 effective samples is usually plenty for a good approximation of the posterior. But we also want to run multiple chains, for reasons we'll discuss in more depth in the next sections. There will be specific advice in Section 9.5 (page 287).

For now, it's worth noting that you can easily parallelize those chains, as well. They can all run at the same time, instead of in sequence. So as long as your computer has four cores (it probably does), it won't take longer to run four chains than one chain. To run four independent Markov chains for the model above, and to distribute them across separate cores in your computer, just increase the number of `chains` and add a `cores` argument:

R code 9.16

```
m9.1 <- ulam(
    alist(
        log_gdp_std ~ dnorm( mu , sigma ) ,
        mu <- a[cid] + b[cid]*( rugged_std - 0.215 ) ,
        a[cid] ~ dnorm( 1 , 0.1 ) ,
        b[cid] ~ dnorm( 0 , 0.3 ) ,
        sigma ~ dexp( 1 )
    ) , data=dat_slim , chains=4 , cores=4 )
```

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

There are a bunch of optional arguments that allow us to tune and customize the process. We'll bring them up as they are needed. For now, keep in mind that `show` will remind you of the model formula and also how long each chain took to run:

R code 9.17
```
show( m9.1 )
```

```
Hamiltonian Monte Carlo approximation
2000 samples from 4 chains

Sampling durations (seconds):
        warmup sample total
chain:1   0.06   0.03  0.09
chain:2   0.05   0.03  0.09
chain:3   0.05   0.03  0.08
chain:4   0.05   0.04  0.10

Formula:
log_gdp_std ~ dnorm(mu, sigma)
mu <- a[cid] + b[cid] * (rugged_std - 0.215)
a[cid] ~ dnorm(1, 0.1)
b[cid] ~ dnorm(0, 0.3)
sigma ~ dexp(1)
```

There were 2000 samples from all 4 chains, because each 1000 sample chain uses by default the first half of the samples to adapt. Something curious happens when we look at the summary:

R code 9.18
```
precis( m9.1 , 2 )
```

```
       mean   sd  5.5% 94.5% n_eff Rhat4
a[1]   0.89 0.02  0.86  0.91  2490     1
a[2]   1.05 0.01  1.03  1.07  3020     1
b[1]   0.13 0.08  0.01  0.25  2729     1
b[2]  -0.14 0.06 -0.23 -0.06  2867     1
sigma  0.11 0.01  0.10  0.12  2368     1
```

If there were only 2000 samples in total, how can we have more than 2000 effective samples for each parameter? It's no mistake. The adaptive sampler that Stan uses is so good, it can actually produce sequential samples that are better than uncorrelated. They are anti-correlated. This means it can explore the posterior distribution so efficiently that it can beat random. It's Jaynes' principle (page 270) in action.

9.4.4. Visualization. By plotting the samples, you can get a direct appreciation for how Gaussian (quadratic) the actual posterior density has turned out to be. Use `pairs` directly on the model object, so that R knows to display parameter names and parameter correlations:

R code 9.19
```
pairs( m9.1 )
```

FIGURE 9.7 shows the resulting plot. This is a pairs plot, so it's still a matrix of bivariate scatter plots. But now along the diagonal the smoothed histogram of each parameter is shown, along

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

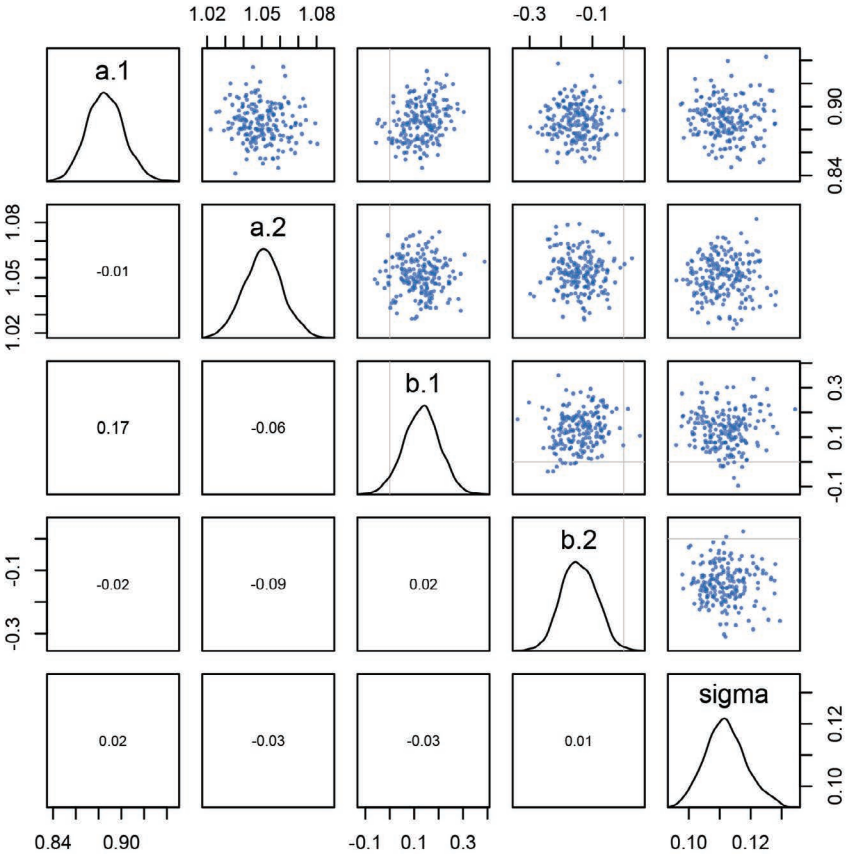


FIGURE 9.7. Pairs plot of the samples produced by `ulam`. The diagonal shows a density estimate for each parameter. Below the diagonal, correlations between parameters are shown.

with its name. And in the lower triangle of the matrix, the correlation between each pair of parameters is shown, with stronger correlations indicated by relative size.

For this model and these data, the resulting posterior distribution is quite nearly multivariate Gaussian. The density for `sigma` is certainly skewed in the expected direction. But otherwise the quadratic approximation does almost as well as Hamiltonian Monte Carlo. This is a very simple kind of model structure of course, with Gaussian priors, so an approximately quadratic posterior should be no surprise. Later, we'll see some more exotic posterior distributions.

9.4.5. Checking the chain. Provided the Markov chain is defined correctly, then it is guaranteed to converge in the long run to the answer we want, the posterior distribution. But some posterior distributions are hard to explore—there will be examples—and the time it would take for them to provide an unbiased approximation is very long indeed. Such problems are rarer for HMC than other algorithms, but they still exist. In fact, one of the virtues

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

of HMC is that it tells us when things are going wrong. Other algorithms, like Metropolis-Hastings, can remain silent about major problems. In the next major section, we'll dwell on causes of and solutions to malfunction.

For now, let's look at two chain visualizations that can often, but not always, spot problems. The first is called a **trace plot**. A trace plot merely plots the samples in sequential order, joined by a line. It's King Markov's path through the islands, in the metaphor at the start of the chapter. Looking at the trace plot of each parameter is often the best thing for diagnosing common problems. And once you come to recognize a healthy, functioning Markov chain, quick checks of trace plots provide a lot of peace of mind. A trace plot isn't the last thing analysts do to inspect MCMC output. But it's often the first.

In the terrain ruggedness example, the trace plot shows a very healthy chain.

R code 9.20
```
traceplot( m9.1 )
```

The result is shown in Figure 9.8 (top). Actually, the figure shows the trace of just the first chain. You can get this by adding `chains=1` to the call. You can think of the zig-zagging trace of each parameter as the path the chain took through each dimension of parameter space. The gray region in each plot, the first 500 samples, marks the *adaptation* samples. During adaptation, the Markov chain is learning to more efficiently sample from the posterior distribution. So these samples are not reliable to use for inference. They are automatically discarded by `extract.samples`, which returns only the samples shown in the white regions of Figure 9.8.

Now, how is this chain a healthy one? Typically we look for three things in these trace plots: (1) stationarity, (2) good mixing, and (3) convergence. Stationarity refers to the path of each chain staying within the same high-probability portion of the posterior distribution. Notice that these traces, for example, all stick around a very stable central tendency, the center of gravity of each dimension of the posterior. Another way to think of this is that the mean value of the chain is quite stable from beginning to end. Good mixing means that the chain rapidly explores the full region. It doesn't slowly wander, but rather rapidly zig-zags around, as a good Hamiltonian chain should. Convergence means that multiple, independent chains stick around the same region of high probability.

Trace plots are a natural way to view a chain, but they are often hard to read, because once you start plotting lots of chains over one another, the plot can look very confusing and hide pathologies in some chains. A second way to visualize the chains is a plot of the distribution of the ranked samples, a **trace rank plot**, or **trank plot**.[155] What this means is to take all the samples for each individual parameter and rank them. The lowest sample gets rank 1. The largest gets the maximum rank (the number of samples across all chains). Then we draw a histogram of these ranks for each individual chain. Why do this? Because if the chains are exploring the same space efficiently, the histograms should be similar to one another and largely overlapping. The `rethinking` package provides a function to produce these:

R code 9.21
```
trankplot( m9.1 )
```

The result is reproduced in Figure 9.8 (bottom). The axes are not labeled in these plots, to reduce clutter. But the horizontal is rank, from 1 to the number of samples across all chains (2000 in this example). The vertical axis is the frequency of ranks in each bin of the

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

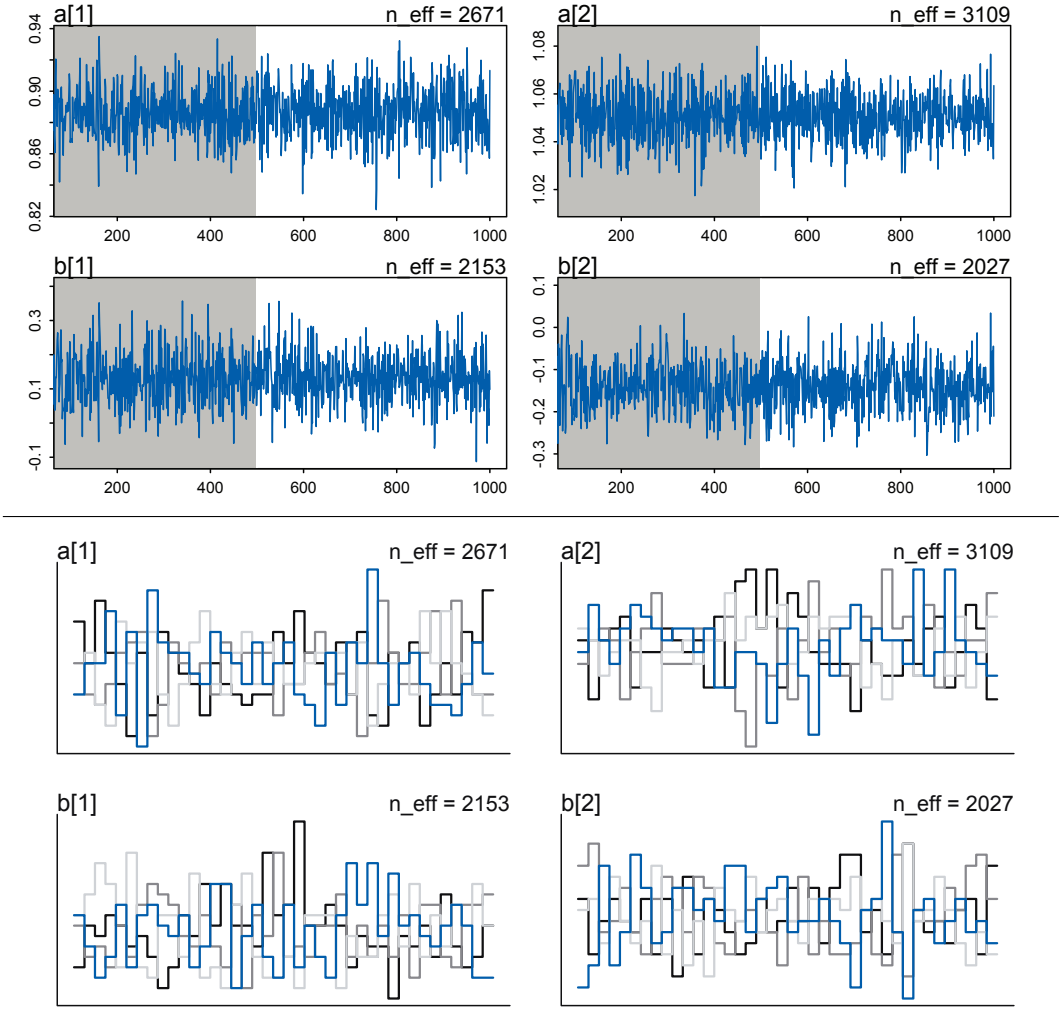


FIGURE 9.8. Trace (top) and trank (bottom) plots of the Markov chain from the ruggedness model, `m9.1`. `sigma` not shown. This is a healthy Markov chain, both stationary and well-mixing. Top: Gray region is warmup.

histogram. This trank plot is what we hope for: Histograms that overlap and stay within the same range.

To really understand the value of these plots, you'll have to see some trace and trank plots for unhealthy chains. That's the project of the next section.

Overthinking: Raw Stan model code. All `ulam` does is translate a list of formulas into Stan's modeling language. Then Stan does the rest. Learning how to write Stan code is not necessary for most of the models in this book. But other models do require some direct interaction with Stan, because it is capable of much more than `ulam` allows you to express. And even for simple models, you'll gain additional comprehension and control, if you peek into the machine. You can always access the raw Stan code that `ulam` produces by using the function `stancode`. For example, `stancode(m9.1)` prints out the Stan code for the ruggedness model. Before you're familiar with Stan's language, it'll look long

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

and weird. But let's take it one piece at a time. It's actually just stuff you've already learned, expressed a little differently.

```
data{
    vector[170] log_gdp_std;
    vector[170] rugged_std;
    int cid[170];
}
parameters{
    vector[2] a;
    vector[2] b;
    real<lower=0> sigma;
}
model{
    vector[170] mu;
    sigma ~ exponential( 1 );
    b ~ normal( 0 , 0.3 );
    a ~ normal( 1 , 0.1 );
    for ( i in 1:170 ) {
        mu[i] = a[cid[i]] + b[cid[i]] * (rugged_std[i] - 0.215);
    }
    log_gdp_std ~ normal( mu , sigma );
}
```

This is Stan code, not R code. It is essentially the formula list you provided to `ulam`, with the implied definitions of the variables made explicit. There are three "blocks."

The first block is the `data` block, at the top. This is where observed variables are named and their types and sizes are declared. `int cid[170]` just means an integer variable named `cid` with 170 values. That's our continent index. The other two are vectors of real values, continuous ruggedness and log GDP variables. Each line in Stan ends in a semicolon. Don't ask why. Just do it. You probably aren't using enough semicolons in your life, anyway.

The next block is `parameters`. These, you can probably guess, are the unobserved variables. They are described just like the observed ones. The new elements here are the `<lower=0>` for `sigma` and those `vector[2]` things. `<lower=0>` tells Stan that `sigma` must be positive. It is constrained. This constraint corresponds to the exponential prior we assign it, which is only defined on the positive reals. The `vector[2]` types are lists of real numbers of length 2. These are our 2 intercepts `a` and our 2 slopes `b`.

If you haven't used explicit and static typed languages before, these first two blocks must seem weird. Why does Stan force the user to say explicitly what R and `ulam` figure out automatically? One reason is that the code doesn't have to do as much checking of conditions, when the types of the variables are already there and unchanging. So it can be faster. But from our perspective, a major advantage is that explicit types help us avoid a large class of programming mistakes. The kinds of run-time shenanigans common to languages like R and Python are impossible in C++. In my experience, people who have studied compiled languages see static typing as a welcome feature. People who have only worked in interpreted languages like R see it as a bother. Both groups are correct.

Finally, the `model` block is where the action is. This block computes the log-probability of the data. It runs from top to bottom, like R code does, adding mathematical terms to the log-probability. So when Stan sees `sigma ~ exponential( 1 )`, it doesn't do any sampling at that moment. Instead, it adds a probability term to the log-probability. This term is just `dexp( sigma , 1 )`. The same goes for the other lines with ~ in them. Note that the last line, for `log_gdp_std`, is vectorized just like R code. There are 170 outcomes values and 170 corresponding `mu` values. That last statement processes all of them.

Stan then uses the analytical gradient—derivative—of all these terms to define the physics simulation under Hamiltonian Monte Carlo. How does Stan do this? It uses a technique called **AUTOMATIC DIFFERENTIATION**, or simply "autodiff," to build an analytical gradient. If you know much about machine learning, you may have also heard about **BACKPROPAGATION**. It's the same thing as autodiff. This is much more accurate than a gradient approximated numerically. If you know some calculus,

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

really all that is going on is ruthless application of the chain rule. But the algorithm is actually quite clever. See the Stan manual for more details.

That's all there is to a Stan program, in the basic case. I'll break out into boxes like this in later chapters, to show more of the raw Stan code. Tools like `ulam` are bridges. They can do a lot of useful work, but the extra control you get from working directly in Stan is worthwhile. Especially since it won't tie you to R or any other specific scripting language.

9.5. Care and feeding of your Markov chain

Markov chain Monte Carlo is a highly technical and usually automated procedure. You might write your own MCMC code, for the sake of learning. But it is very easy to introduce subtle biases. A package like Stan, in contrast, is continuously tested against expected output. Most people who use Stan don't really understand what it is doing, under the hood. That's okay. Science requires division of labor, and if every one of us had to write our own Markov chains from scratch, a lot less research would get done in the aggregate.

But as with many technical and powerful procedures, it's natural to feel uneasy about MCMC and maybe even a little superstitious. Something magical is happening inside the computer, and unless we make the right sacrifices and say the right words, an ancient evil might awake. So we do need to understand enough to know when the evil stirs. The good news is that HMC, unlike Gibbs sampling and ordinary Metropolis, makes it easy to tell when the magic goes wrong. Its best feature is not how efficient it is. Rather the best feature is that it complains loudly when things aren't right. Let's look at some complaints and along the way establish some guidelines for running chains.

9.5.1. How many samples do you need? You can control the number of samples from the chain by using the `iter` and `warmup` parameters. The defaults are 1000 for `iter` and `warmup` is set to `iter/2`, which gives you 500 warmup samples and 500 real samples to use for inference. But these defaults are just meant to get you started, to make sure the chain gets started okay. Then you can decide on other values for `iter` and `warmup`.

So how many samples do we need for accurate inference about the posterior distribution? It depends. First, what really matters is the *effective* number of samples, not the raw number. The effective number of samples is an estimate of the number of independent samples from the posterior distribution, in terms of estimating some function like the posterior mean. Markov chains are typically *autocorrelated*, so that sequential samples are not entirely independent. This happens when chains explore the posterior slowly, like in a Metropolis algorithm. Autocorrelation reduces the effective number of samples. Stan provides an estimate of effective number of samples, for the purpose of estimating the posterior mean, as `n_eff`. You can think of `n_eff` as the length of a Markov chain with no autocorrelation that would provide the same quality of estimate as your chain. One consequence of this definition, as you saw earlier in the chapter, is that `n_eff` can be larger than the length of your chain, provided sequential samples are anti-correlated in the right way. While `n_eff` is only an estimate, it is usually better than the raw number of samples, which can be very misleading.

Second, what do you want to know? If all you want are posterior means, it doesn't take many samples at all to get very good estimates. Even a couple hundred samples will do. But if you care about the exact shape in the extreme tails of the posterior, the 99th percentile or so, then you'll need many more. So there is no universally useful number of samples to aim for. In most typical regression applications, you can get a very good estimate of the posterior

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

mean with as few as 200 effective samples. And if the posterior is approximately Gaussian, then all you need in addition is a good estimate of the variance, which can be had with one order of magnitude more, in most cases. For highly skewed posteriors, you'll have to think more about which region of the distribution interests you. Stan will sometimes warn you about "tail ESS," the effective sample size (similar to `n_eff`) in the tails of the posterior. In those cases, it is nervous about the quality of extreme intervals, like 95%. Sampling more usually helps.

The warmup setting is more subtle. On the one hand, you want to have the shortest warmup period necessary, so you can get on with real sampling. But on the other hand, more warmup can mean more efficient sampling. With Stan models, typically you can devote as much as half of your total samples, the `iter` value, to warmup and come out very well. But for simple models like those you've fit so far, much less warmup is really needed. Models can vary a lot in the shape of their posterior distributions, so again there is no universally best answer. But if you are having trouble, you might try increasing the warmup. If not, you might try reducing it. There's a practice problem at the end of the chapter that guides you in experimenting with the amount of warmup.

Rethinking: Warmup is not burn-in. Other MCMC algorithms and software often discuss BURN-IN. With a sampling strategy like ordinary Metropolis, it is conventional and useful to trim off the front of the chain, the "burn-in" phase. This is done because it is unlikely that the chain has reached stationarity within the first few samples. Trimming off the front of the chain hopefully removes any influence of which starting value you chose for a parameter.[156]

But Stan's sampling algorithms use a different approach. What Stan does during warmup is quite different from what it does after warmup. The warmup samples are used to adapt sampling, to find good values for the step size and the number of steps. Warmup samples are not representative of the target posterior distribution, no matter how long warmup continues. They are not burning in, but rather more like cycling the motor to heat things up and get ready for sampling. When real sampling begins, the samples will be immediately from the target distribution, assuming adaptation was successful.

9.5.2. How many chains do you need? It is very common to run more than one Markov chain, when estimating a single model. To do this with `ulam` or `stan` itself, the `chains` argument specifies the number of independent Markov chains to sample from. And the optional `cores` argument lets you distribute the chains across different processors, so they can run simultaneously, rather than sequentially. All of the non-warmup samples from each chain will be automatically combined in the resulting inferences.

So the question naturally arises: How many chains do we need? There are three answers to this question. First, when initially debugging a model, use a single chain. There are some error messages that don't display unless you use only one chain. The chain will fail with more than one chain, but the reason may not be displayed. This is why the `ulam` default is `chains=1`. Second, when deciding whether the chains are valid, you need more than one chain. Third, when you begin the final run that you'll make inferences from, you only really need one chain. But using more than one chain is fine, as well. It just doesn't matter, once you're sure it's working. I'll briefly explain these answers.

The first time you try to sample from a chain, you might not be sure whether the chain is working right. So of course you will check the trace plot or trank plot. Having more than one chain during these checks helps to make sure that the Markov chains are all converging to the same distribution. Sometimes, individual chains look like they've settled down to a stable

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

distribution, but if you run the chain again, it might settle down to a different distribution. When you run multiple Markov chains, each with different starting positions, and see that all of them end up in the same region of parameter space, it provides a check that the machine is working correctly. Using 3 or 4 chains is often enough to reassure us that the sampling is working properly.

But once you've verified that the sampling is working well, and you have a good idea of how many warmup samples you need, it's perfectly safe to just run one long chain. For example, suppose we learn that we need 1000 warmup samples and about 9000 real samples in total. Should we run one chain, with `warmup=1000` and `iter=10000`, or rather 3 chains, with `warmup=1000` and `iter=4000`? It doesn't really matter, in terms of inference. But it might matter in efficiency, because the 3 chains duplicate warmup effect that just gets thrown away. And since warmup is typically the slowest part of the chain, these extra warmup samples cost a disproportionate amount of your computer's time. On the other hand, if you run the chains on different processor cores, then you might prefer 3 chains, because you can spread the load and finish the whole job faster. My institute uses shared computing servers with 64 or more available cores. We run a lot of parallel chains.

There are exotic situations in which all of the advice above must be modified. But for typical regression models, you can live by the motto *one short chain to debug, four chains for verification and inference*.

Things may still go wrong. One of the perks of using HMC and Stan is that when sampling isn't working right, it's usually very obvious. As you'll see in the sections to follow, bad chains tend to have conspicuous behavior. Other methods of MCMC sampling, like Gibbs sampling and ordinary Metropolis, aren't so easy to diagnose.

Rethinking: Convergence diagnostics. The default diagnostic output from Stan includes two metrics, `n_eff` and `Rhat`. The first is a measure of the effective number of samples. The second is the Gelman-Rubin convergence diagnostic, $\hat{R}$.[157] When `n_eff` is much lower than the actual number of iterations (minus warmup) of your chains, it means the chains are inefficient, but possibly still okay. When `Rhat` is above 1.00, it usually indicates that the chain has not yet converged, and probably you shouldn't trust the samples. If you draw more iterations, it could be fine, or it could never converge. See the Stan user manual for more details. It's important however not to rely too much on these diagnostics. Like all heuristics, there are cases in which they provide poor advice. For example, `Rhat` can reach 1.00 even for an invalid chain. So view it perhaps as a signal of danger, but never of safety. For conventional models, these metrics typically work well.

9.5.3. Taming a wild chain. One common problem with some models is that there are broad, flat regions of the posterior density. This happens most often, as you might guess, when one uses flat priors. The problem this can generate is a wild, wandering Markov chain that erratically samples extremely positive and extremely negative parameter values.

Let's look at a simple example. The code below tries to estimate the mean and standard deviation of the two Gaussian observations −1 and 1. But it uses totally flat priors.

R code 9.22

```
y <- c(-1,1)
set.seed(11)
m9.2 <- ulam(
    alist(
        y ~ dnorm( mu , sigma ) ,
```

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

```
        mu <- alpha ,
        alpha ~ dnorm( 0 , 1000 ) ,
        sigma ~ dexp( 0.0001 )
    ) , data=list(y=y) , chains=3 )
```

Now let's look at the `precis` output:

R code 9.23

```
precis( m9.2 )
```

```
        mean      sd    5.5%   94.5% n_eff Rhat
alpha  69.38  393.89 -363.57  739.53   116 1.03
sigma 568.53 1247.18    6.81 2563.38   179 1.02
```

Whoa! This posterior can't be right. The mean of −1 and 1 is zero, so we're hoping to get a mean value for `alpha` around zero. Instead we get crazy values and implausibly wide intervals. Inference for `sigma` is no better. The `n_eff` and `Rhat` diagnostics don't look good either. We drew 1500 samples total, but the estimated effective sample sizes are 116 and 179. You might get different numbers, but they will qualitatively be just as bad.

You should also see several warning messages, including:

```
Warning messages:
1: There were 67 divergent transitions after warmup. Increasing adapt_delta
above 0.95 may help. See
http://mc-stan.org/misc/warnings.html#divergent-transitions-after-warmup
```

There is useful advice at the URL. The quick version is that Stan detected problems in exploring all of the posterior. These are **DIVERGENT TRANSITIONS**. I'll give a more thorough explanation in a later chapter. Think of them as Stan's way of telling you there are problems with the chains. For simple models, increasing the `adapt_delta` control parameter will usually remove the divergent transitions. This is explained more in the Overthinking box at the end of this section. You can try adding `control=list(adapt_delta=0.99)` to the `ulam` call—`ulam`'s default is 0.95. But it won't help much in this specific case. This problem runs deeper, with the model itself.

You should also see a second warning:

```
2: Examine the pairs() plot to diagnose sampling problems
```

This refers to Stan's `pairs` method, not `ulam`'s. To use it, try `pairs( m9.2@stanfit )`. This is like `ulam`'s pairs plot, but divergent transitions are colored in red. For that reason, the plot won't reproduce in this book. So be sure to inspect it on your own machine. The shape of the posterior alone should shake your confidence.

Now take a look at the trace plot for this fit, `traceplot(m9.2)`. It's shown in the top row of FIGURE 9.9. The reason for the weird estimates is that the Markov chains seem to drift around and spike occasionally to extreme values. This is not a healthy pair of chains, and they do not provide useful samples. The `trankplot(m9.2)` is also shown. The rank histograms spend long periods with one chain above or below the others. This indicates poor exploration of the posterior.

It's easy to tame this particular chain by using weakly informative priors. The reason the model above drifts wildly in both dimensions is that there is very little data, just two observations, and flat priors. The flat priors say that every possible value of the parameter is equally plausible, a priori. For parameters that can take a potentially infinite number of

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

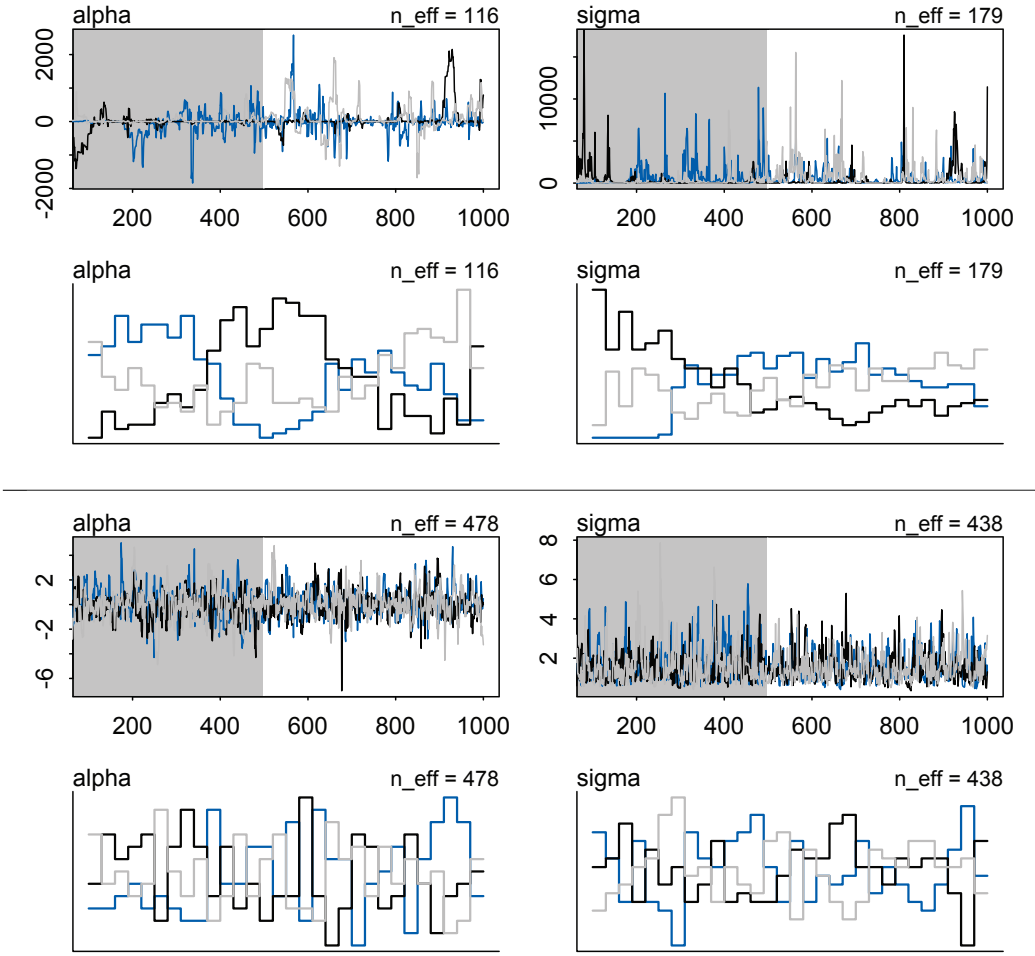


FIGURE 9.9. Diagnosing and healing a sick Markov chain. Top two rows: Trace and trank plots from three chains defined by model `m9.2`. These chains are not healthy. Bottom two rows: Adding weakly informative priors in `m9.3` clears up the condition right away.

values, like `alpha`, this means the Markov chain needs to occasionally sample some pretty extreme and implausible values, like negative 30 million. These extreme drifts overwhelm the chain. If the likelihood were stronger, then the chain would be fine, because it would stick closer to zero.

But it doesn't take much information in the prior to stop this foolishness, even without more data. Let's use this model:

$$
\begin{aligned}
y_i &\sim \text{Normal}(\mu, \sigma) \\
\mu &= \alpha \\
\alpha &\sim \text{Normal}(1, 10) \\
\sigma &\sim \text{Exponential}(1)
\end{aligned}
$$

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

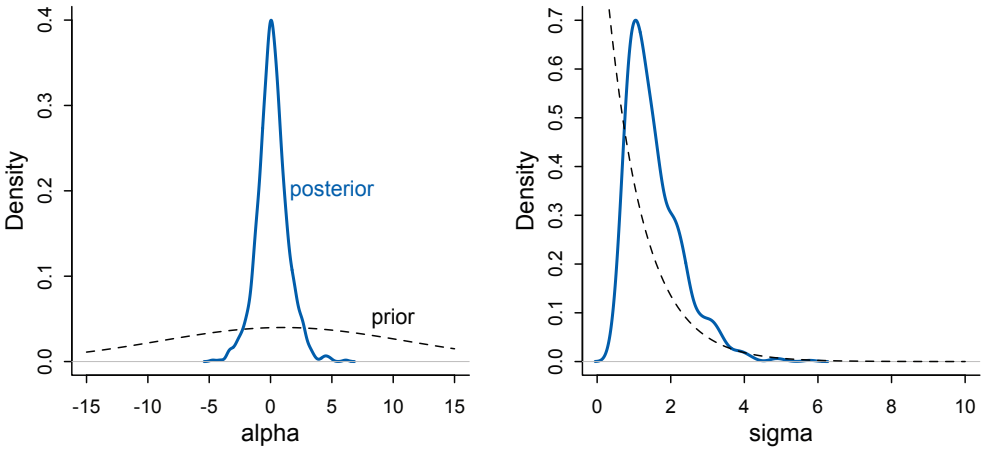


FIGURE 9.10. Prior (dashed) and posterior (blue) for the model with weakly informative priors, `m9.3`. Even with only two observations, the likelihood easily overcomes these priors. Yet the posterior cannot be successfully approximated without them.

I've just added weakly informative priors for α and σ. We'll plot these priors in a moment, so you will be able to see just how weak they are. But let's re-approximate the posterior first:

R code 9.24

```
set.seed(11)
m9.3 <- ulam(
    alist(
        y ~ dnorm( mu , sigma ) ,
        mu <- alpha ,
        alpha ~ dnorm( 1 , 10 ) ,
        sigma ~ dexp( 1 )
    ) , data=list(y=y) , chains=3 )
precis( m9.3 )
```

```
      mean   sd  5.5% 94.5% n_eff Rhat4
alpha 0.10 1.13 -1.60  1.97   478     1
sigma 1.52 0.72  0.67  2.86   438     1
```

That's much better. Take a look at the bottom portion of FIGURE 9.9. The trace and trank plots look healthy. Both chains are stationary around the same values, and mixing is good. No more wild detours into the thousands. And those divergent transitions are gone.

To appreciate what has happened, take a look at the priors (dashed) and posteriors (blue) in FIGURE 9.10. Both the Gaussian prior for α and the exponential prior for σ contain very gradual downhill slopes. They are so gradual, that even with only two observations, as in this example, the likelihood almost completely overcomes them. The mean of the prior for α is 1, but the mean of the posterior is zero, just as the likelihood says it should be.

These weakly informative priors have helped by providing a very gentle nudge towards reasonable values of the parameters. Now values like 30 million are no longer equally plausible as small values like 1 or 2. Lots of problematic chains want subtle priors like these,

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

designed to tune estimation by assuming a tiny bit of prior information about each parameter. And even though the priors end up getting washed out right away—two observations were enough here—they still have a big effect on inference, by allowing us to get an answer. That answer is also a good answer. This point will be even more important for non-Gaussian models to come.

Rethinking: The folk theorem of statistical computing. The example above illustrates Andrew Gelman's FOLK THEOREM OF STATISTICAL COMPUTING: When you have computational problems, often there's a problem with your model.[158] Before we begin to tune the software and pour more computer power into a problem, it can be useful to go over the model specification again, and the data itself, to make sure the problem isn't in the pre-sampling stage. It's very common when working with Bayesian models that slow or clunky sampling is due to something as simple as having entirely omitted one or more prior distributions.

Overthinking: Divergent transitions are your friend. You'll see DIVERGENT TRANSITION warnings often in using `ulam` and Stan. They are your friend, providing a helpful warning. These warnings arise when the numerical simulation that HMC uses is inaccurate. HMC can detect these inaccuracies. That is one of its major advantages over other sampling approaches, most of which provide few automatic ways to discover bad chains. We'll examine these divergent transitions in much more detail in a later chapter. We'll also see some clever ways to work around them.

9.5.4. Non-identifiable parameters. Back in Chapter 6, you met the problem of highly correlated predictors and the non-identifiable parameters they can create. Here you'll see what such parameters look like inside of a Markov chain. You'll also see how you can identify them, in principle, by using a little prior information. Most importantly, the badly behaving chains produced in this example will exhibit characteristic bad behavior, so when you see the same pattern in your own models, you'll have a hunch about the cause.

To construct a non-identifiable model, we first simulate 100 observations from a Gaussian distribution with mean zero and standard deviation 1.

R code 9.25

```
set.seed(41)
y <- rnorm( 100 , mean=0 , sd=1 )
```

By simulating the data, we know the right answer. Then we fit this model:

$$
\begin{aligned}
y_i &\sim \text{Normal}(\mu, \sigma) \\
\mu &= \alpha_1 + \alpha_2 \\
\alpha_1 &\sim \text{Normal}(0, 1000) \\
\alpha_2 &\sim \text{Normal}(0, 1000) \\
\sigma &\sim \text{Exponential}(1)
\end{aligned}
$$

The linear model contains two parameters, α_1 and α_2, which cannot be identified. Only their sum can be identified, and it should be about zero, after estimation.

Let's run the Markov chain and see what happens. This chain is going to take much longer than the previous ones. But it should still finish after a few minutes.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

R code 9.26

```
set.seed(384)
m9.4 <- ulam(
    alist(
        y ~ dnorm( mu , sigma ) ,
        mu <- a1 + a2 ,
        a1 ~ dnorm( 0 , 1000 ),
        a2 ~ dnorm( 0 , 1000 ),
        sigma ~ dexp( 1 )
    ) , data=list(y=y) , chains=3 )
precis( m9.4 )
```

```
          mean     sd    5.5%  94.5% n_eff Rhat4
a1    -364.81 318.57 -792.82 240.50     2  2.80
a2     365.00 318.57 -240.26 792.98     2  2.80
sigma    1.05   0.10    0.90   1.19     2  2.02
```

Those estimates look suspicious, and the `n_eff` and `Rhat` values are terrible. The means for `a1` and `a2` are about the same distance from zero, but on opposite sides of zero. And the standard deviations are massive. This is a result of the fact that we cannot simultaneously estimate `a1` and `a2`, but only their sum. You should also see a warning:

```
Warning messages:
1: There were 1199 transitions after warmup that exceeded the maximum treedepth.
Increase max_treedepth above 10. See
http://mc-stan.org/misc/warnings.html#maximum-treedepth-exceeded
```

This is confusing. If you visit the URL, you'll see that this means the chains are inefficient, because some internal limit was reached. These treedepth warnings usually indicate inefficient chains, but not necessarily broken chains. To increase the treedepth, you can add `control=list(max_treedepth=15)` to the `ulam` call. But it won't help much. There is something else seriously wrong here.

Looking at the trace plot reveals more. The left column in FIGURE 9.11 shows two Markov chains from the model above. These chains do not look like they are stationary, nor do they seem to be mixing very well. Indeed, when you see a pattern like this, it is reason to worry. Don't use these samples.

Again, weakly regularizing priors can rescue us. Now the model fitting code is:

R code 9.27

```
m9.5 <- ulam(
    alist(
        y ~ dnorm( mu , sigma ) ,
        mu <- a1 + a2 ,
        a1 ~ dnorm( 0 , 10 ),
        a2 ~ dnorm( 0 , 10 ),
        sigma ~ dexp( 1 )
    ) , data=list(y=y) , chains=3 )
precis( m9.5 )
```

```
      mean   sd   5.5% 94.5% n_eff Rhat4
a1    0.01 7.16 -11.43 11.54   389     1
a2    0.18 7.15 -11.41 11.57   389     1
sigma 1.03 0.08   0.92  1.17   448     1
```

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

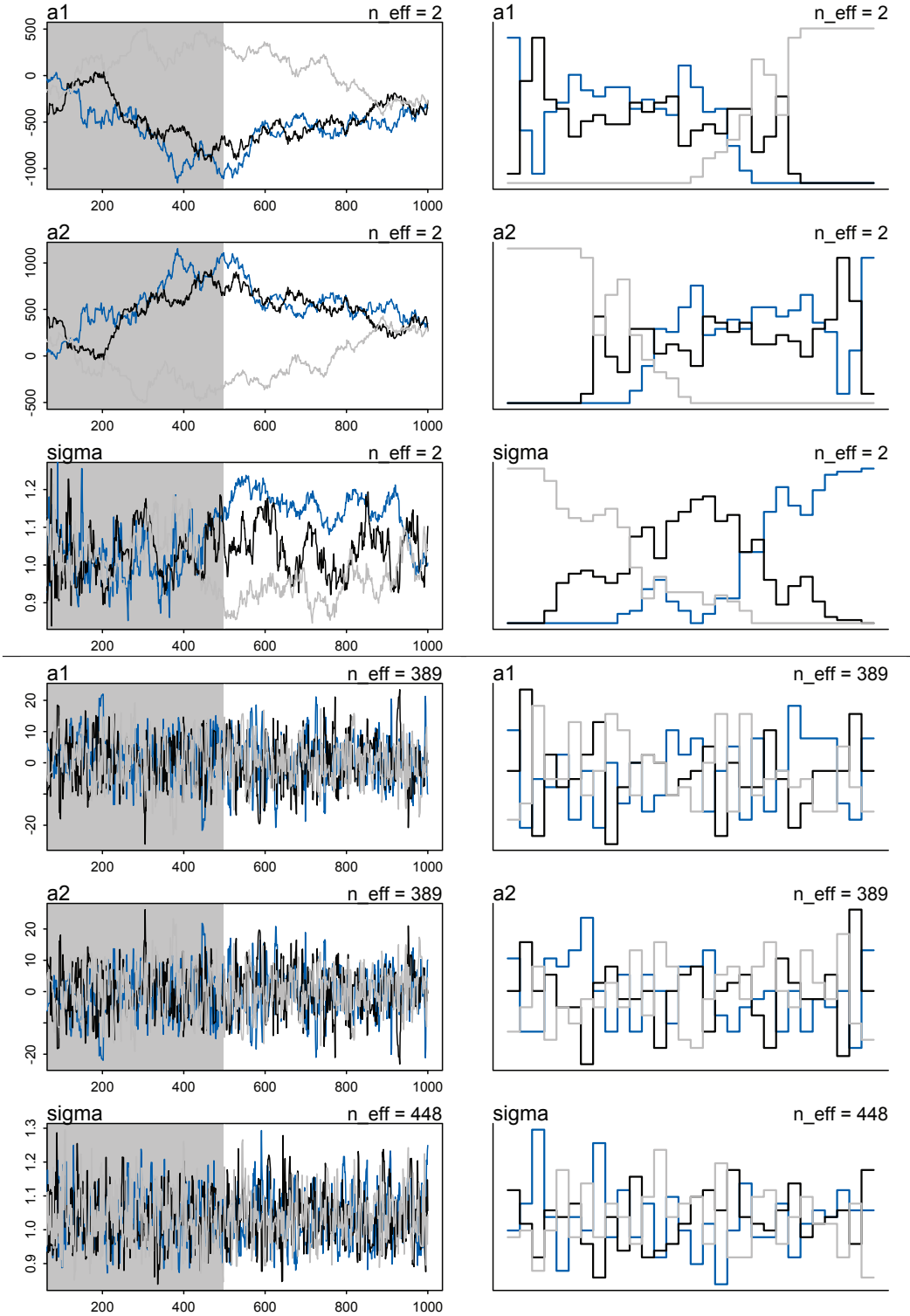


FIGURE 9.11. Top panel, `m9.4`. A chain with wandering parameters, `a1` and `a2`. Bottom panel, `m9.5`. Same model but with weakly informative priors.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

The estimates for `a1` and `a2` are better identified now. Well, they still aren't individually identified. But their sum is identified. Compare the trace and trank plots in FIGURE 9.11. Notice also that the model sampled a lot faster. With flat priors, `m9.4`, sampling may take 3 times as long as it does for `m9.5`. Often, a model that is very slow to sample is under-identified. This is an aspect of the **FOLK THEOREM OF STATISTICAL COMPUTING** (page 293).

In the end, adding some weakly informative priors saves this model. You might think you'd never accidentally try to fit an unidentified model. But you'd be wrong. Even if you don't make obvious mistakes, complex models can easily become unidentified or nearly so. With many predictors, and especially with interactions, correlations among parameters can be large. Just a little prior information telling the model "none of these parameters can be 30 million" often helps, and it has no effect on estimates. A flat prior really is flat, all the way to infinity. Unless you believe infinity is a reasonable estimate, don't use a flat prior.

Additionally, adding weak priors can speed up sampling, because the Markov chain won't feel that it has to run out to extreme values that you, but not your model, already know are highly implausible.

Rethinking: Hamiltonian warnings and Gibbs overconfidence. When people start using Stan, or some other Hamiltonian sampler, they often find that models they used to fit in Metropolis-Hastings and Gibbs samplers like BUGS, JAGS, and MCMCglmm no longer work well. The chains are slow. There are lots of warnings. Stan is really something of a nag. Is something wrong with Stan?

No. Those problems were probably always there, even in the other tools. But since Gibbs doesn't use gradients, it doesn't notice some issues that a Hamiltonian engine will. A culture has evolved in applied statistics of running bad chains for a very long time—for millions of iterations—and then thinning aggressively, praying, and publishing. Phylogenetic analyses may be particularly prone to this, since tree spaces are very difficult to explore.[159] Tools like Stan and other Hamiltonian engines are so important for reliable research precisely because they provide more diagnostic criteria for the accuracy of the Monte Carlo approximation. Don't resent the nagging.

9.6. Summary

This chapter has been an informal introduction to Markov chain Monte Carlo (MCMC) estimation. The goal has been to introduce the purpose and approach MCMC algorithms. The major algorithms introduced were the Metropolis, Gibbs sampling, and Hamiltonian Monte Carlo algorithms. Each has its advantages and disadvantages. The `ulam` function in the `rethinking` package was introduced. It uses the Stan (mc-stan.org) Hamiltonian Monte Carlo engine to fit models as they are defined in this book. General advice about diagnosing poor MCMC fits was introduced by the use of a couple of pathological examples. In the next chapters, we use this new power to learn new kinds of models.

9.7. Practice

Problems are labeled Easy (**E**), Medium (**M**), and Hard (**H**).

9E1. Which of the following is a requirement of the simple Metropolis algorithm?

(1) The parameters must be discrete.
(2) The likelihood function must be Gaussian.
(3) The proposal distribution must be symmetric.

9E2. Gibbs sampling is more efficient than the Metropolis algorithm. How does it achieve this extra efficiency? Are there any limitations to the Gibbs sampling strategy?

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

9E3. Which sort of parameters can Hamiltonian Monte Carlo not handle? Can you explain why?

9E4. Explain the difference between the effective number of samples, `n_eff` as calculated by Stan, and the actual number of samples.

9E5. Which value should `Rhat` approach, when a chain is sampling the posterior distribution correctly?

9E6. Sketch a good trace plot for a Markov chain, one that is effectively sampling from the posterior distribution. What is good about its shape? Then sketch a trace plot for a malfunctioning Markov chain. What about its shape indicates malfunction?

9E7. Repeat the problem above, but now for a trace rank plot.

9M1. Re-estimate the terrain ruggedness model from the chapter, but now using a uniform prior for the standard deviation, `sigma`. The uniform prior should be `dunif(0,1)`. Use `ulam` to estimate the posterior. Does the different prior have any detectible influence on the posterior distribution of `sigma`? Why or why not?

9M2. Modify the terrain ruggedness model again. This time, change the prior for `b[cid]` to `dexp(0.3)`. What does this do to the posterior distribution? Can you explain it?

9M3. Re-estimate one of the Stan models from the chapter, but at different numbers of `warmup` iterations. Be sure to use the same number of sampling iterations in each case. Compare the `n_eff` values. How much warmup is enough?

9H1. Run the model below and then inspect the posterior distribution and explain what it is accomplishing.

R code 9.28

```
mp <- ulam(
    alist(
        a ~ dnorm(0,1),
        b ~ dcauchy(0,1)
    ), data=list(y=1) , chains=1 )
```

Compare the samples for the parameters `a` and `b`. Can you explain the different trace plots? If you are unfamiliar with the Cauchy distribution, you should look it up. The key feature to attend to is that it has no expected value. Can you connect this fact to the trace plot?

9H2. Recall the divorce rate example from Chapter 5. Repeat that analysis, using `ulam` this time, fitting models `m5.1`, `m5.2`, and `m5.3`. Use `compare` to compare the models on the basis of WAIC or PSIS. To use WAIC or PSIS with `ulam`, you need add the argument `log_log=TRUE`. Explain the model comparison results.

9H3. Sometimes changing a prior for one parameter has unanticipated effects on other parameters. This is because when a parameter is highly correlated with another parameter in the posterior, the prior influences both parameters. Here's an example to work and think through.

Go back to the leg length example in Chapter 6 and use the code there to simulate height and leg lengths for 100 imagined individuals. Below is the model you fit before, resulting in a highly correlated posterior for the two beta parameters. This time, fit the model using `ulam`:

R code 9.29

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.

```
m5.8s <- ulam(
    alist(
        height ~ dnorm( mu , sigma ) ,
        mu <- a + bl*leg_left + br*leg_right ,
        a ~ dnorm( 10 , 100 ) ,
        bl ~ dnorm( 2 , 10 ) ,
        br ~ dnorm( 2 , 10 ) ,
        sigma ~ dexp( 1 )
    ) , data=d, chains=4,
    start=list(a=10,bl=0,br=0.1,sigma=1) )
```

Compare the posterior distribution produced by the code above to the posterior distribution produced when you change the prior for `br` so that it is strictly positive:

R code 9.30

```
m5.8s2 <- ulam(
    alist(
        height ~ dnorm( mu , sigma ) ,
        mu <- a + bl*leg_left + br*leg_right ,
        a ~ dnorm( 10 , 100 ) ,
        bl ~ dnorm( 2 , 10 ) ,
        br ~ dnorm( 2 , 10 ) ,
        sigma ~ dexp( 1 )
    ) , data=d, chains=4,
    constraints=list(br="lower=0"),
    start=list(a=10,bl=0,br=0.1,sigma=1) )
```

Note the `constraints` list. What this does is constrain the prior distribution of `br` so that it has positive probability only above zero. In other words, that prior ensures that the posterior distribution for `br` will have no probability mass below zero. Compare the two posterior distributions for `m5.8s` and `m5.8s2`. What has changed in the posterior distribution of both beta parameters? Can you explain the change induced by the change in prior?

9H4. For the two models fit in the previous problem, use WAIC or PSIS to compare the effective numbers of parameters for each model. You will need to use `log_lik=TRUE` to instruct `ulam` to compute the terms that both WAIC and PSIS need. Which model has more effective parameters? Why?

9H5. Modify the Metropolis algorithm code from the chapter to handle the case that the island populations have a different distribution than the island labels. This means the island's number will not be the same as its population.

9H6. Modify the Metropolis algorithm code from the chapter to write your own simple MCMC estimator for globe tossing data and model from Chapter 2.

9H7. Can you write your own Hamiltonian Monte Carlo algorithm for the globe tossing data, using the R code in the chapter? You will have to write your own functions for the likelihood and gradient, but you can use the `HMC2` function.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:26:22.